

How Missing Data Hurts and Imputation Helps across Typical AI tasks: An Empirical Evaluation of Data Quality-Driven Imputation Methods: [Experiments & Analysis]

Anonymous authors

ABSTRACT

With the rise of data-centric AI, understanding how missing data hurts model utility and how imputation helps restore it has become critical, yet existing evaluations remain limited to reconstruction metrics and single tasks. To address this, we propose **IIKP-Path**, a four-stage empirical framework that systematically deconstructs the chain from *Imputation* to *Data Quality*, *Key Factors*, and *Model Performance* across typical AI tasks. Using MLPs as sensitive probes, we evaluate 13 representative data quality-driven imputation methods, revealing that imputation efficacy is not universal but contingent on the alignment between algorithmic mechanisms and task-specific data structures. We introduce PG (Performance Gap), a unified metric that quantifies relative performance loss, enabling fair cross-task comparison, and identify *Key Factors* that govern downstream utility more effectively than generic quality metrics. We propose **KFBF** (Key-Factors Boundary Finder) algorithms to map the feasible bounds of factor interactions, and provide *13 empirical findings*, while demonstrating that sensitivity patterns from MLP probes transfer robustly to complex architectures (e.g., TSMixer). Accordingly, we develop a four-layer, task-aware selection strategy. Powered by a pruning-based recommendation algorithm, it reduces non-viable candidates by 7.92% and cuts inference time by 53.49% compared to exhaustive search. This work provides actionable guidance for building robust, data-quality-aware AI systems.

1 INTRODUCTION

The rapid development of AI is profoundly driving changes in many key fields. While breakthroughs in deep learning (DL) and large-scale models continue to push performance frontiers, growing evidence suggests that the primary bottlenecks in real-world AI systems often stem not merely from algorithmic limitations, but fundamentally from the quality deficiencies of data [62]. Poor-quality data can severely undermine model generalization, prediction reliability, and overall system trustworthiness [45, 81, 93]. As the *fuel* of AI pipelines, **data quality** has emerged as a central concern of data-centric AI. High-quality data is not merely beneficial but essential. Recent empirical studies consistently demonstrate that models trained on clean small datasets can outperform those trained on large-scale, noisy ones while consuming fewer resources [24, 76, 98]. Consequently, data quality management is no longer the auxiliary preprocessing step before training, but the indispensable component of AI system design [67].

Among data quality challenges, missing values are particularly pervasive and consequential. In practice, missingness commonly arises from sensor malfunctions, privacy constraints, or sampling biases [47, 53, 61], with observed missing rates frequently ranging from 20% to over 80% [47, 53]. Moreover, the underlying missing mechanisms are often complex and non-ignorable [10]. Current

AI workflows typically address missing data through coarse pre-processing techniques, e.g., sample deletion, mean or forward filling, or the application of generic, task-agnostic imputation tools. While operationally convenient, these methods represent little more than *engineering compromises*! They offer no guarantee that the imputed data will preserve the statistical dependencies, decision boundaries, or temporal patterns critical to downstream model performance [42, 54, 67].

In contrast, the database community has been deeply engaged in missing-value imputation and data cleaning for years, developing a diverse spectrum of techniques ranging from classical statistical to advanced machine learning and deep learning approaches [2, 46, 61]. Recent SIGMOD works have further expanded this, introducing specialized solutions for photovoltaic data [55], time series [108], in-database execution [90], and uncertainty-aware imputation [34]. Despite these advances, the majority of existing studies remain predominantly focused on *data recovery accuracy*, with limited evaluation of how imputed data actually impacts downstream AI model utility. A pervasive assumption across many methods is that minimizing reconstruction error (e.g., RMSE) inherently translates to improved task performance [34, 46, 55, 61, 90, 108]. However, from an AI-centric perspective, this assumption frequently breaks down. Recent empirical studies in clinical classification and time series forecasting demonstrate that missingness can severely distort temporal dynamics, disrupt feature learning, resulting in significant performance degradation even after sophisticated imputation [51, 56, 78]. Consequently, the pivotal question for AI practitioners is no longer *whether missing entries can be numerically recovered*, but rather *whether the repaired data effectively preserve the structural and semantic properties necessary to support downstream task learning*.

Against this backdrop, the central objective of this paper is to systematically investigate *how missing data hurts and how imputation helps across AI tasks*. We move beyond merely assessing the extent to which imputation improves surface-level data completeness, aiming instead to rigorously analyze its real impact on the utility of downstream models. Although recent initiatives (e.g., CleanML [52] and REIN [1]), advocate for the co-optimization of data quality and downstream tasks, empirical understanding of how missing values and diverse imputation strategies influence model performance across heterogeneous tasks remains limited. Specifically, we identify four critical challenges: (1) **What to measure?** Task sensitivity to missingness varies widely, yet conventional data quality metrics are task-agnostic and data-centric. Balancing generality with task specificity remains a methodological bottleneck. (2) **How they relate?** Imputation strategy, data properties, and model performance form a highly coupled system. But no formal framework exists to quantify how specific quality changes translate into performance shifts or uncovering their interactions. (3) **Why it happens?** Imputation effects propagate through multiple intermediate stages, while

existing studies lack fine-grained decomposition, which requires well-defined analytical stages, interpretable proxy metrics, and a correlation model. All of these are currently absent. (4) **What to do?** Practical recommendations still rely on expert heuristics or black-box meta-learners, lacking interpretability and adaptability. Effective guidance demands integrating task-aware quality metrics, impact pathways, and correlation models into explainable decision rules.

To answer these questions, our work builds upon three strands of prior research: ① **imputation strategy & data quality**. No universally optimal imputation method exists, and the *best* choice depends critically on the evaluation metrics used [29, 34, 36, 42, 46, 54, 55, 78, 108]. ② **data quality & model performance**. General quality metrics often fail to predict downstream performance [46, 78]. In contrast, task-aware studies offer deeper insights, e.g., [71] analyzes how noise distribution affects classification, [13, 92] emphasize time series decomposition and model sensitivity to trend and seasonality. ③ **imputation & performance**. Missingness itself harms performance [62], and the degradation intensifies as missing rates increase [61]. Collectively, these works indicate that the true AI value of imputation methods can be rigorously assessed when they are evaluated within concrete, task-specific contexts.

To address these gaps, we pose two central questions: **RQ1**. *How do imputation methods change data quality and thereby affect model performance across diverse analytics tasks?* and **RQ2**. *Do these effects exhibit interpretable, predictable, and transferable patterns?* To delve deeper into the impact mechanisms, we employ the Multilayer Perceptron (MLP) as a controlled experimental probe, justified by three properties: ① **Universality**. MLPs serve as fundamental building blocks in DL architectures (e.g., TSMixer [11] and MLP-Mixer [86]). ② **Sensitivity**. Lacking built-in robustness mechanisms (e.g., attention or residual connections), it readily amplifies data perturbations induced by imputation, making latent effects more observable. ③ **Comparability**. As a minimal, architecture-agnostic baseline, the MLP effectively isolates the impact of data quality from model complexity. The obtained laws can provide guidance for complex (MLP-derived) models.

To the best of our knowledge, this work is among the earliest to deconstruct the coupling among *Imputation*, *Data Quality*, and *Model Performance* through an association pathway lens. Unlike single-task studies [47, 78], we conduct evaluation across three major tasks. Compared to evaluations relying solely on reconstruction accuracy or end-to-end performance [61], we introduce a multi-dimensional assessment framework. Furthermore, in contrast to prior works that incorporate multiple factors but lack task-oriented grounding [46], we extend data quality via task-aware key factors, revealing the intrinsic mechanisms governing how imputation impacts downstream utility. Our **contributions** are as follows:

(1) We construct **IIKP-Path**, a **four-stage analytical pathway**, *Imputation* $\rightarrow$ *Imputation quality* $\rightarrow$ *Key factors* $\rightarrow$ *Performance*, which integrates 13 representative imputation methods, 3 categories of quality metrics, interpretable key factors, and 3 typical tasks (classification, regression, and time series forecasting.) into a unified experimental pipeline. It achieves systematic validation of imputation assumptions, task-aware extension of data quality via key factors, and quantification and ranking of each factor's contribution to performance changes.

(2) We introduce **Key-Factors Boundary Finder (KBBF)**, an algorithm that computes the feasible bounds of a task key factors through four coordinated functions, enabling fine-grained exploration of performance inflection points and provides a mechanistic lens to analyze factor interactions. Further, we propose **Performance Gap (PG)**, a unified metric that quantifies the relative performance loss induced by imputation, normalized against oracle performance on complete data. PG eliminates task-specific scale differences, and supports consistent, cross-task, cross-dataset, and cross-method comparison.

(3) We derive three-layer mechanistic findings from IIKP-Path: ① **Imputation** $\rightarrow$ **Quality**. No imputation method is universally optimal, and their effectiveness depends on quality metrics and manifests differently across task-specific key factors. ② **Quality** $\rightarrow$ **Performance**. Post-imputation quality correlates with, but does not determine, model performance. Outcomes are governed by dominant factors (e.g., class discriminability in classification), whose interactions define a task-aware notion of data quality. ③ **Imputation** $\rightarrow$ **Model Behavior**. While MLPs exhibit consistent sensitivity to missingness, degradation pathways diverge by task. Crucially, we validate that findings from MLP probes transfer to two additional base models and generalize to 9 MLP-derived deep architectures (e.g., TSMixer, LightTS), supporting lightweight probes as a scalable strategy for guiding complex model deployment.

(4) We propose four-layer **task-aware imputation selection strategy**. ① *Data-algorithm pre-screening* based on data characteristics (e.g., missing rate) and computational budget, ② *Quality-assisted filtering* to retain statistically superior imputations, ③ *Task-key factor alignment* by matching imputation behavior to dominant task-specific factors, and ④ *Optimization guidance & Risk warnings*. Providing actionable advice for high-missingness regimes. Furthermore, we design a pruning recommendation method that, over 10-13 candidate methods, achieves $\geq 78.40\%$ coverage (within ≤ 3 rank positions of the true best) of the optimal performance, reduces non-viable imputations by 7.92%, and cuts inference time by 53.49% versus exhaustive search. It outperforms conventional single-metric or heuristic strategies. All code, results and the extended technical report are available at <https://anonymous.4open.science/r/IIKP-Path-BE65> to ensure reproducibility.

Organization. Section 2 discusses related work. Section 4 details the design of IIKP-Path. Section 5 derives key findings. Section 6 provides task-aware imputation recommendations, and Section 7 draws the conclusion.

2 RELATED WORK

Missing data imputation. Current mainstream methods can be categorized into statistical, similarity- [41, 73], tree- [12, 80], regression- [75, 105], compression- [50, 60], and DL-based [31, 101] imputation. Existing research primarily follows two directions: (1) **surveys** and **benchmarks** [46, 61] that compare these methods based on data type, missingness mechanism (i.e., Missing Completely At Random MCAR, Missing At Random MAR, Missing Not At Random MNAR), or review domain-specific techniques (e.g., time series [2, 47], transportation [109], medicine [4]), and (2) **methodological innovations**, such as MPIN [53] for multi-attribute sensor

streams, realistic missing pattern simulation [37], and optimized imputation order [106].

MLP, as a foundational probe model, serves as a foundational building block for diverse ML tasks, including signal classification [64, 72], crowd-density estimation [23], and electricity-load forecasting [77]. In recent years, MLP-based architectures have emerged as prominent competitors in time series forecasting [11, 20, 70, 91, 100, 103, 107] and computer vision [22, 35, 86]. Furthermore, hybrid designs integrating MLPs with CNNs [102] or RNNs [43] demonstrate their versatility in recommendation and intrusion detection systems to combine complementary strengths and enhance model capacity.

AI-oriented data quality and Data-centric AI. The rise of data-centric AI has emphasized systematic data engineering as a foundation for building robust AI systems [104]. Surveys have categorized and analyzed core data tasks [79, 104], with data quality emerging as a central concern. Recent work increasingly examines how data defects affect model behavior [62, 93, 112]. Within this context, data preprocessing for ML has gained significant attention. REIN [1] evaluates the impact of cleaning in ML pipelines, while COMET [63] proposes a cost-aware, feature-level cleaning recommender. CleanML [82] aim to make preprocessing effects on ML transparent. Among these, imputation plays a critical role. Theoretical studies question its necessity, e.g., [110] introduces *certain models* that can handle missingness without imputation [110]. Empirical works, however, highlight its practical impact: [26, 94] analyze how imputation choices affect classification performance and interpretability, and [78] proposes task-aware imputation metrics to bridge data quality and model outcomes. Despite these efforts, a systematic, cross-task evaluation framework that connects imputation to downstream performance via interpretable key factors remains absent, motivating our proposed IIKP-Path.

3 PRELIMINARIES

3.1 Datasets

To support systematic empirical analysis, we define four formalized datasets that characterize the complete processing pipeline from raw data to imputed inputs.

① **High-quality complete dataset** D_{clean} , serves as the experimental baseline, derived from real world datasets for classification, regression, and time series forecasting tasks (detailed in Section 5.1). At this stage, all samples are free of missing values, representing an ideal data quality state.

② **Missing dataset** D_{miss} , generated by artificially injecting missing values into D_{clean} according to predefined missing rates and missing mechanisms, simulating incomplete data in real world.

③ **Imputed dataset** $D_{\text{imp}}^{(k)}$, obtained by applying the k -th imputation algorithm to D_{miss} .

④ **Processed dataset** D_{pro} , obtained through controlled interventions on D_{clean} or D_{miss} according to predefined procedures (e.g., perturbing target components of D_{clean} for time series forecasting), aiming to investigate the influence of specific factors on model behavior.

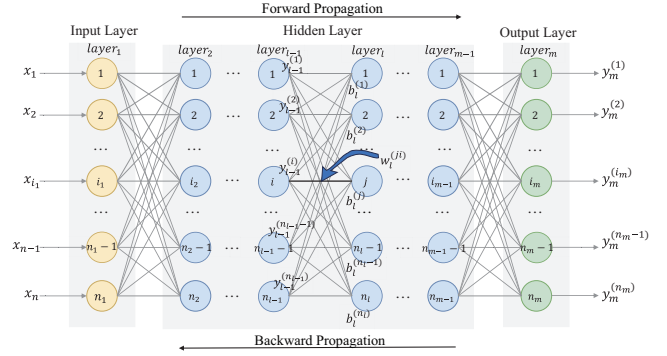


Figure 1: The architecture of MLP.

3.2 Imputation algorithms

To systematically assess the impact of different imputation methods, we select 13 mainstream imputation algorithms spanning six major categories. These methods represent widely adopted technical routes in current practice, and their core principles are summarized as follows.

① **Statistical methods.** Include mean, median, and mode, which replace missing values with the central tendency of the attribute.

② **Similarity-based methods.** K-Nearest Neighbor (KNN) [73] and Hot Deck Imputation (HDI) [41], infer missing entries by identifying observations similar to the incomplete sample.

③ **Tree-based methods.** MissForest Imputation (MissFI) [80] and XGBoost Imputation (XGBI) [12] leverage ensemble tree models to iteratively predict missing values. MissFI is based on random forests, while XGBI builds on the gradient boosting framework.

④ **Compression-based methods.** Soft Impute (SI) [60] reconstructs the data matrix via soft-thresholded singular value decomposition (SVD), while Matrix Factorization Imputation (MFI) [50] learns explicit low-rank latent representations.

⑤ **Regression-based methods.** Multivariate imputation by Chained Equations (MICE) [75] performs multiple imputation through iterative conditional regression models. Imputation via Individual Model (IIM) [105] selects similar samples via KNN for each attribute and integrates predictions from their optimal regressors.

⑥ **DL-based methods.** Generative Adversarial Imputation Network (GAIN) [101] employs a generative adversarial mechanism, while Multiple Imputation Denoising Autoencoder (MIDAE) [31] implements multiple imputation using a denoising autoencoder (DAE) with random initializations.

All algorithms are implemented according to their original papers or widely adopted open-source versions, and are uniformly tuned and evaluated in our experiments (Section 5).

3.3 The Multilayer Perceptron model (MLP)

We adopt the Multilayer Perceptron (MLP) as an effective *mediator* model to study the impact of task-centric missing data imputation. An MLP is a feedforward neural network containing an input layer, an output layer, and one or more hidden layers. Each neuron (i.e., node) in the hidden and output layers applies a nonlinear activation function to a weighted sum of its inputs. The network is trained via supervised learning techniques, typically employing backpropagation in conjunction with gradient descent optimization.

As shown in Figure 1, for a MLP with m layers, each neuron j in the l -th layer ($2 \leq l \leq m$) receives a weighted sum of outputs from the previous layer,

$$u_l^{(j)} = \sum_{i \in \text{layer}_{l-1}} w_l^{(ji)} y_{l-1}^{(i)} + b_l^{(j)}.$$

Its output is

$$y_l^{(j)} = f(u_l^{(j)}),$$

where $f(\cdot)$ is a nonlinear activation function (e.g. ReLU or Tanh), and $w_l^{(ji)}$ and $b_l^{(j)}$ represent the connection weights and biases, respectively. The network is trained end-to-end by minimizing the loss function $\mathcal{L}$, with the gradient calculated using the backpropagation algorithm. Define the error term $\delta_l^{(j)} = \frac{\partial \mathcal{L}}{\partial u_l^{(j)}}$, and its recursive relationship is

$$\delta_l^{(j)} = \begin{cases} f'(u_l^{(j)}) \sum_{k \in \text{layer}_{l+1}} \delta_{l+1}^{(kj)} w_{l+1}^{(kj)}, & 1 \leq l < m, \\ f'(u_l^{(j)}) \frac{\partial \mathcal{L}}{\partial y_l^{(j)}}, & l = m, \end{cases}$$

The parameter update rule is

$$w_l^{(ji)} \leftarrow w_l^{(ji)} - \eta \delta_l^{(j)} y_{l-1}^{(i)}, \quad b_l^{(j)} \leftarrow b_l^{(j)} - \eta \delta_l^{(j)}.$$

3.4 Downstream tasks

To systematically evaluate the impact of the aforementioned imputation methods on model performance, we select three canonical data analytics tasks, classification, regression, and time series forecasting. These three tasks all have well-defined formalizations in terms of input-output structure, learning objectives, and loss functions, and all can be modeled by MLP effectively.

(1) **Classification.** Given an input feature vector $x \in \mathbb{R}^d$ and a corresponding class label $y \in \{1, 2, \dots, K\}$, the goal is to learn a mapping $f: \mathbb{R}^d \rightarrow \{1, \dots, K\}$ that minimizes the prediction error. MLP outputs a class probability $\hat{p} = \text{softmax}(z) \in \Delta^{K-1}$ through the softmax output layer, where z is the linear output of the last layer. Training typically uses cross-entropy loss:

$$\mathcal{L}_{\text{cls}} = - \sum_{k=1}^K \mathbb{I}(y = k) \log \hat{p}_k.$$

The objective of classification tasks is to categorize input data into predefined classes correctly. From early linear discriminant functions [27] and decision trees, to kernel-based SVM [17], and finally to various neural networks, the core issue is how to construct feature representations and classification surfaces with stronger discriminative power and better generalization ability. Although modern classification models mostly adopt architectures such as CNN and Transformer, their core still relies on the learning of nonlinear discriminant boundaries. As a basic nonlinear classifier, MLP's sensitivity to feature perturbations makes it an ideal carrier for studying class discriminability, providing insights for other network architectures that establish decisive boundaries similarly.

(2) The **regression** task aims to learn the mapping $f: \mathbb{R}^d \rightarrow \mathbb{R}$ from feature vector $x \in \mathbb{R}^d$ to continuous target $y \in \mathbb{R}$. The MLP directly outputs a scalar prediction $\hat{y} = f(x)$ and optimizes it using

mean squared error (MSE) or mean absolute error (MAE) loss:

$$\mathcal{L}_{\text{reg}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad \text{or} \quad \mathcal{L}_{\text{reg}} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|.$$

Its methodological development has always focused on improving fitting accuracy and generalization ability. Due to its sensitivity to distribution shifts or correlation disruptions caused by imputation, regression serves as a key vehicle for evaluating how imputation affects continuous mapping fidelity. MLP, with its function approximation capability, perform regression as one of its most basic and core tasks.

(3) Let the time series be $\{x_t\}_{t=1}^T$, the objective of **time series forecasting** is to predict the future τ step values $X_{\text{fut}} = [x_{T+1}, \dots, x_{T+\tau}] \in \mathbb{R}^\tau$ based on a historical window $X_{\text{hist}} = [x_{T-H+1}, \dots, x_T] \in \mathbb{R}^H$. MLP models it as a regression problem: $\hat{X}_{\text{fut}} = f(X_{\text{hist}})$, and the loss function is also MSE or MAE. For time series forecasting, traditional methods include ARIMA [7], Exponential Smoothing, Spectral Analysis [49], etc., while advanced models mainly based on Transformer [58, 68, 96], CNN [57, 95], RNN [33], MLP [11, 91, 107]. The advantage of MLP architecture lies in its ability for global context aggregation and modeling of channel independence, making it highly sensitive to the fidelity of components such as trend and seasonality. Therefore, this paper adopts the MLP as the foundational probing model and validates the transferability of the obtained patterns to the aforementioned deep MLP variants in Section 5.6.

The MLP configurations for the three tasks mentioned above are summarized in Table 1, covering input format, output format, loss function, and the role of MLP and representative architectures, providing a standardized basis for subsequent unified experimental evaluation.

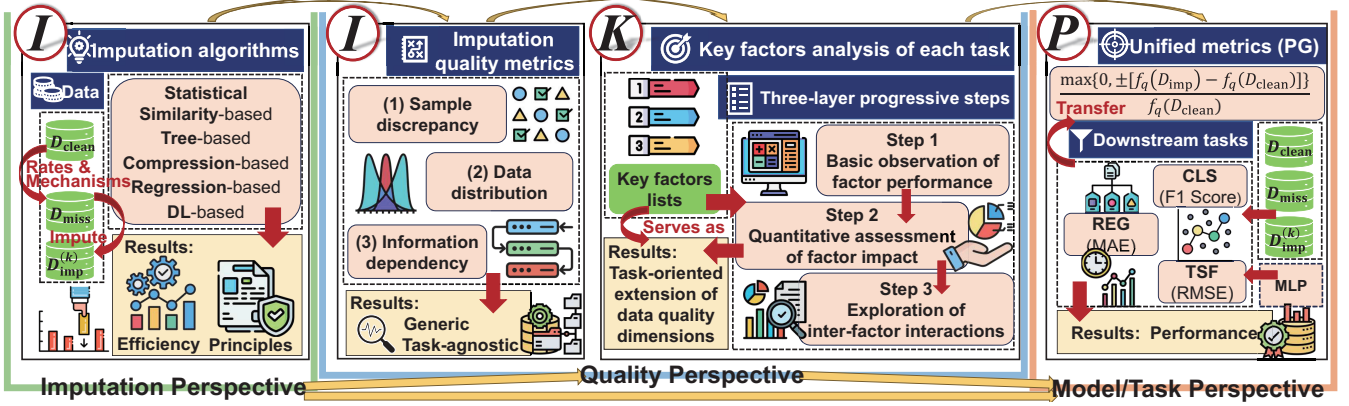
4 METHODOLOGY

To systematically and deeply investigate the impact mechanisms of missing data imputation on downstream task performance, we propose and implement the IIKP-Path, as shown in Figure 2. Our evaluation framework built around three perspectives: ① Imputation perspective (green boxes), characterizes the principle and properties of imputation algorithms. ② Quality perspective (blue boxes), combines traditional imputation quality assessment with task oriented-extensions via key factors analysis. ③ Model/Task perspective (orange boxes), quantifies the ultimate impact by applying MLP on classification, regression, and time series forecasting tasks and measuring their performance.

Guided by this, we conduct a systematic analysis along three directions. ① **Impact of imputation on data quality**, examined through (a) *imputation algorithm analysis* $\rightarrow$ *imputation quality assessment*, measuring the effect of imputation on distribution and information (corresponding to *Finding 1* and *Finding 2*); and (b) *imputation algorithm analysis* $\rightarrow$ *key factors analysis*, examining the performance and underlying reasons of imputation on task-oriented quality dimensions (corresponding to *Finding 5*, *Finding 8*, and *Finding 10*). ② **Association between data quality and task performance**, analyzed via (a) *imputation quality assessment* $\rightarrow$ *performance evaluation*, verifying whether general quality metrics can predict performance (corresponding to *Finding 4*); and (b) *key*

Table 1: Standardized configuration of MLP in three downstream tasks

Tasks	Input format	Output format	Loss function	The role of MLP and representative architectures
CLS	Feature vector $x \in \mathbb{R}^d$	Class probability distribution $\hat{p} \in \Delta^{K-1}$ (through Softmax)	Cross-entropy loss: $\mathcal{L} = -\sum_{k=1}^K \mathbb{I}(y=k) \log \hat{p}_k$	Basic nonlinear discriminator; representative architectures: ResMLP [87], Dynamic MLP [97] for image classification, and Graph-MLP [40] for graph node classification.
REG	Feature vector $x \in \mathbb{R}^d$	Scalar prediction $\hat{y} \in \mathbb{R}$	Distance-based loss functions such as MSE, MAE	Function approximation benchmark; the most direct application of MLP, serves to quantify the perturbation that imputation introduces to continuous mapping relationships.
TSF	Historical window $X_{\text{hist}} \in \mathbb{R}^{H \times C}$	Future series $\hat{X}_{\text{fut}} \in \mathbb{R}^{r \times C}$	MSE / MAE (point-wise or sequence-level)	DL models based on MLP are currently one of the mainstream methods for time series forecasting, with representative architectures including TSMixer, LightTS, and TimeMixer.

**Figure 2: The overview of IIKP-Path.**

factors analysis $\rightarrow$ performance evaluation, revealing the dominant role and interaction mechanisms of task-specific factors on performance (corresponding to Finding 6, Finding 7, Finding 9, Finding 11, and Finding 12). ③ **Patterns of imputation effects on models and tasks.** (corresponding to Finding 3).

4.1 Imputation algorithm analysis

As the initial stage of the IIKP-Path, this section provides a systematic dissection of 13 representative imputation algorithms, focusing on characterizing their essential differences by their *information utilization patterns* and *underlying assumptions*, thus deducing their applicable scenarios. Starting from high quality complete dataset D_{clean} , we inject missing values under the MCAR mechanism at missing rates $\alpha \in \{5\%, 10\%, \dots, 95\%\}$ to generate D_{miss} , then apply each method to produce the corresponding imputed dataset $D_{\text{imp}}^{(k)}$. While imputation efficiency (runtime) and empirical verification are reported in Section 5.2, here we focus on theoretical analysis.

The essence of imputation lies in making reasonable estimations of unknown information based on known information. The core distinction among different imputation algorithms lies in the definition and utilization of known information and the estimation method of the unknown information.

(1) **Statistical methods (Mean, Median, Mode).** This method defines known information as global statistical measures. Mean, Median, and Mode represent the solutions that minimize Mean Squared Error (MSE), minimize Mean Absolute Error (MAE), and Maximum Likelihood Estimation (MLE), respectively. They are computationally efficiency and simple to implement. However, they ignore feature dependency and local structures, and thus cannot

capture any additional information, easily leading to relational ignorance bias. Consequently, their applicability is relatively limited, only suitable for scenarios with low missing rates and datasets with approximately independent features.

(2) **Similarity-based methods (KNN, HDI).** This method assumes that *similar samples exhibit similar values in missing attributes*. It estimates missing values by weighting local neighbors. Let x_{j_i} denote the j -th nearest neighbors of x_i , $j = 1, 2, \dots, k$, then $\hat{x}_i = \sum_{j=1}^k w_j x_{j_i}$, thus the spatial distribution of samples determines the reliability of using x_{j_i} to estimate x_i . Consequently, its effectiveness highly depends on the density of the local structure: When known samples are densely distributed in the vicinity of the target sample, i.e., distances are small, x_{j_i} and x_i exhibit higher similarity, leading to more accurate estimation. Conversely, in high-dimensional sparse or heterogeneous data, the assumption credibility weakens, leading to estimation biases. This will be validated in Section 5.2.

(3) **Tree-based methods (MissFI, XGBI).** This method defines known information as the complex relationships within the data. MissFI integrates decision trees, where the splitting rules commonly use the Gini index or variance calculation: $Gini(I) = (\frac{|I_L|}{|I|} Gini(I_L) + \frac{|I_R|}{|I|} Gini(I_R))$ and $Var(I) = (\frac{|I_L|}{|I|} Var(I_L) + \frac{|I_R|}{|I|} Var(I_R))$, where I denotes the sample sets at the current node, and I_L and I_R represent the sample sets in the left and right child nodes after splitting, satisfying $I = I_L \cup I_R$. XGBI is based on a gradient boosting framework, where the gain before and after splitting is given by $\frac{1}{2} [\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda}]$, where g_i is the first-order gradient of sample i , h_i is the second-order gradient, and λ is the

regularization coefficient. When these methods construct decision trees, each split only considers the samples within the current node, making full use of local data information; during overall integration and gradient boosting, the global data distribution is comprehensively considered. Due to their effective utilization of both global and local structures, they can handle data with complex dependencies relatively well. However, node splitting tends to favor features that yield higher gain, which may cause such methods to over-reliance on several dominant attributes while neglecting other potentially informative ones.

(4) **Compression-based methods (SI, MFI)**. This method assumes that the data possesses a low dimensional intrinsic structure. Their imputation performance is closely tied to the validity of their underlying assumptions. SI is based on a low-rank assumption. For the current matrix $\hat{\mathbf{X}}^{(t)}$, suppose it has M singular values, denoted by $\sigma_1, \sigma_2, \dots, \sigma_M$, where σ_i represents the i -th singular value. During each iteration, σ_i is updated as $\max(\sigma_i - \lambda, 0)$, forming a new matrix $\hat{\mathbf{X}}^{(t+1)}$. The error introduced by the soft-thresholding operation is given by $\|\hat{\mathbf{X}}^{(t)} - \hat{\mathbf{X}}^{(t+1)}\|_F^2 = \sum_{\sigma_i > \lambda} \lambda^2 + \sum_{\sigma_i \leq \lambda} \sigma_i^2$. Arranging the singular values in descending order and approximating the sequence with a simplified model $\sigma_i = Ci^{-\alpha}$ ($\alpha > 0$), where $C \in \mathbb{R}$ is a constant, the rate of decay is controlled by α . When $\alpha \rightarrow \infty$, the decay is extremely rapid, indicating that the low-rank assumption holds well; conversely, when $\alpha \rightarrow 0$, the assumption is not satisfied. The error can thus be expressed in integral form $\int_1^{k_0} \lambda^2 di + \int_{k_0}^M (Ci^{-\alpha})^2 di$, where k_0 satisfies $Ck_0^{-\alpha} = \lambda$. ① Then $\|P_\Omega(\mathbf{X}) - P_\Omega(\hat{\mathbf{X}})\|_F^2 = \lambda^2(k_0 - 1) + \frac{C^2}{1-2\alpha}(M^{1-2\alpha} - k_0^{1-2\alpha})$. Since $M > k_0$, $\lim_{\alpha \rightarrow 0} \frac{C^2}{1-2\alpha}(M^{1-2\alpha} - k_0^{1-2\alpha}) > 0$. Because $Ck_0^{-\alpha} = \lambda$ and $k_0 \geq 1$, $\frac{C}{\lambda} > 1$, then $\lim_{\alpha \rightarrow 0} \lambda^2(k_0 - 1) = \lim_{\alpha \rightarrow 0} \lambda^2 \left[\left(\frac{C}{\lambda} \right)^{\frac{1}{\alpha}} - 1 \right] = \infty$. Therefore, $\lim_{\alpha \rightarrow 0} \|P_\Omega(\mathbf{X}) - P_\Omega(\hat{\mathbf{X}})\|_F^2 = \infty$, with maximum error. ② Substitute $k_0 = \left(\frac{C}{\lambda} \right)^{\frac{1}{\alpha}}$, $\|P_\Omega(\mathbf{X}) - P_\Omega(\hat{\mathbf{X}})\|_F^2 = \lambda^2 \left[\left(\frac{C}{\lambda} \right)^{\frac{1}{\alpha}} - 1 \right] + \frac{C^2}{1-2\alpha} (M^{1-2\alpha} - \left(\frac{C}{\lambda} \right)^{\frac{1}{\alpha}-2})$, that is, $\|P_\Omega(\mathbf{X}) - P_\Omega(\hat{\mathbf{X}})\|_F^2 = \frac{C^2}{1-2\alpha} M^{1-2\alpha} - \lambda^2 + C^2 \left(\frac{C}{\lambda} \right)^{\frac{1}{\alpha}-2} \left(1 - \frac{1}{1-2\alpha} \right)$. Therefore, $\lim_{\alpha \rightarrow \infty} \|P_\Omega(\mathbf{X}) - P_\Omega(\hat{\mathbf{X}})\|_F^2 = 0 - \lambda^2 + C^2 \left(\frac{C}{\lambda} \right)^{-2} = 0$, with minimum error. It will be validated in Section 5.2.

MFI is based on a linear latent factor assumption, to approximate the original matrix through the latent dimension p , i.e., $\mathbf{X} \in \mathbb{R}^{n \times d}$, where $\mathbf{U} \in \mathbb{R}^{n \times p}$ and $\mathbf{V} \in \mathbb{R}^{d \times p}$. If nonlinear relationships exist, i.e., $\mathbf{X} = f(\mathbf{U}, \mathbf{V})$ with $f(\cdot)$ being a nonlinear function, a significant discrepancy arises between $f(\mathbf{U}, \mathbf{V})$ and $\mathbf{UV}^T$. This gap cannot be fully compensated by optimizing only the linear mapping $\mathbf{UV}^T$. Therefore, MFI performs well only when the data conforms to the linear latent factor assumption.

(5) **Regression-based methods (MICE, IIM)**. This method defines known information as the dependencies among variables. The basic formulation treats the variable with missing values Y_j as the target, while the remaining variables Y_{-j} serve as features, fitting regression models $Y_j = g(Y_{-j}; \theta_j) + \epsilon_j$, where θ_j denotes the model parameters, and ϵ_j is the error term. Consequently, such methods can flexibly capture multivariate dependencies. However, the iterative process of MICE may lead to error propagation, where

modeling errors in one step are chained to the next. Meanwhile, the local modeling of IIM may fragment predictions under non-stationary relationships.

(6) **DL-based methods (GAIN, MIDAE)**. This method also defines known information as the complex nonlinear relationships among data. Such methods typically employ deep network architectures with a large number of parameters. According to the VC dimension theory [88], their effective training relies on sufficient observed data. GAIN is based on a generative adversarial network (GAN) composed of a generator and a discriminator. During imputation, the discriminator implicitly learns the global data distribution by distinguishing real values from generated ones; simultaneously, if the generated results violate local structures, they can be easily identified by the discriminator, thereby forcing the generator to balance both global distribution matching and local pattern fidelity. However, when the missing rate is high, specific patterns in the remaining data may dominate, causing the generator to produce only a single pattern or few patterns, i.e., mode collapse occurs.

MIDAE builds upon DAE, whose training objective is to minimize the reconstruction error between the noisy input $\mathbf{X} + \epsilon$ and the original data $\mathbf{X}$: $\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{X}, \epsilon} [\|\mathbf{X} - f_\theta(\mathbf{X} + \epsilon)\|^2]$, where f_θ is the mapping function of the DAE and θ denotes its parameters. This method can fully exploit both distributional information and local structures of the data. However, during optimization, the DAE tends to prioritize recovery of high frequency or large amplitude components, while providing weaker gradient updates for smooth or low frequency signals, leading to a phenomenon of reconstruction preference bias.

4.2 Data quality assessment

Data-centric AI research [32, 112] has profoundly clarified that data quality is a foundational determinant of model performance. Existing imputation quality evaluation approaches fall into three groups: ① **Sample discrepancy** measures pointwise similarity between D_{clean} and D_{imp} using metrics such as MSE, R^2 , and Pearson correlation coefficient (ρ). ② **Data distribution** evaluates recovery of marginal or joint distribution per feature, using metrics such as KL divergence, KS test, and 2-Wasserstein distance [85], or Sliced Wasserstein distance [78], which better handles high-dimensional data. ③ **Information dependency** quantifies preservation of statistical dependencies via metrics like mutual information.

However, current imputation evaluation frameworks [2, 61] still rely excessively on sample discrepancy measurement, making it difficult to comprehensively reflect how imputation preserves and recovers data distribution and information. To address this, this paper adopts all three measurements to construct a multi-dimensional assessment. While sample discrepancy is integrated with task-specific evaluation in Section 4.4, we focus here on distributional and dependency-based measures, and establish the *Imputation algorithm analysis* $\rightarrow$ *imputation quality assessment* pipeline.

(1) **Kolmogorov-Smirnov (KS) test**. It examines whether two samples originate from the same distribution. It is performed on D_{clean} and $D_{\text{imp}}^{(k)}$, let $\hat{F}_{\text{clean}}(\cdot)$ and $\hat{F}_{\text{imp}}^{(k)}(\cdot)$ denote their respective

empirical distribution functions.

$$D_{KS} = \max_{t \in D_{\text{clean}} \cup D_{\text{imp}}^{(k)}} \left| \hat{F}_{\text{clean}}(t) - \hat{F}_{\text{imp}}^{(k)}(t) \right|$$

The p-value (with a significance level τ) is then used to assess whether the imputed data can maintain the same distribution as the clean data. If $p < \tau$ suggests no significant distributional difference. The KS test is computationally simple and sensitive to local discrepancy. However, it focuses on maximum local deviation but ignores overall distribution shifts.

(2) **Kullback-Leibler (KL) divergence**. It measures the difference between probability distributions. Let $p_{\text{imp}}^{(k)}$ and p_{clean} denote the probability density functions of $D_{\text{imp}}^{(k)}$ and D_{clean} , calculate the KL divergence of $D_{\text{imp}}^{(k)}$ relative to D_{clean} :

$$D_{KL}(p_{\text{imp}}^{(k)} || p_{\text{clean}}) = \sum_x p_{\text{imp}}^{(k)}(x) \log \frac{p_{\text{imp}}^{(k)}(x)}{p_{\text{clean}}(x)}$$

which quantifies the difference in probability distribution between the imputed data and the clean data. A smaller KL divergence indicates a smaller difference. While KL divergence can effectively discriminate between probability distributions, it requires substantial distribution overlap. When overlap is minimal, i.e., when there exists an x such that $p_{\text{imp}}^{(k)}$ or p_{clean} approaches zero, the KL divergence becomes meaningless.

(3) **2-Wasserstein distance**. Based on optimal transport theory, the 2-Wasserstein distance quantifies the minimum *movement cost* required to transform $D_{\text{imp}}^{(k)}$ into D_{clean} . Let $P_{\text{imp}}^{(k)}$ and P_{clean} denote the probability distributions of $D_{\text{imp}}^{(k)}$ and D_{clean} , Π be the set of all joint distributions whose marginals are $P_{\text{imp}}^{(k)}$ and P_{clean} , γ represents a joint distribution form Π , then calculate

$$W_2(P_{\text{imp}}^{(k)}, P_{\text{clean}}) = \left[\inf_{\gamma \in \Pi(P_{\text{imp}}^{(k)}, P_{\text{clean}})} \mathbb{E}_{(x,y) \sim \gamma} \|x - y\|^2 \right]^{\frac{1}{2}}$$

which quantifies the cost required to transform the imputed data into the clean data. A smaller 2-Wasserstein distance indicates a lower cost and hence closer distributions. 2-Wasserstein distance does not become invalid due to insufficient distribution overlap. However, it reflects the overall transport cost and is less sensitive to local discrepancy.

(4) **Sliced Wasserstein distance (SWD)**. It is an efficient approximation of the classical Wasserstein distance, specifically designed for evaluating the quality of imputed data. Its core idea is to transform a high-dimensional distribution into multiple one-dimensional Wasserstein distances via random projections, thereby balancing computational efficiency and sensitivity to multidimensional structure. The calculation steps are: First, samples are randomly partitioned into two subsets, I_p and J_p . If the total number of samples is even, I_p and J_p have the same size; otherwise, they differ by one. A random unit direction vector $\vec{n}_r$ is drawn. Next, calculate data projections: the clean data are projected as $x_i \cdot \vec{n}_r$, and the imputed data as $\hat{x}_j \cdot \vec{n}_r$. Finally, calculate SWD: for each combination of r , p , calculate the standard deviation s of $x_i \cdot \vec{n}_r$. The baseline slice distance, i.e., the 2-Wasserstein distance between the two subsets

of D_{clean} , is given by

$$W_{\text{clean}}(r, p) = W_2\left(\frac{x_i \cdot \vec{n}_r}{s}, \frac{x_j \cdot \vec{n}_r}{s}\right), i \in I_p, j \in J_p$$

The imputed slice distance, i.e., the 2-Wasserstein distance between subset I_p of D_{clean} and subset J_p of $D_{\text{imp}}^{(k)}$, is

$$W_{\text{imp}}^{(k)}(r, p) = W_2\left(\frac{x_i \cdot \vec{n}_r}{s}, \frac{\hat{x}_j \cdot \vec{n}_r}{s}\right), i \in I_p, j \in J_p$$

Then, aggregate the results over all (r, p) to obtain the SWD. Following [78], we analyze the distribution of $\frac{W_{\text{imp}}^{(k)}(r, p)}{W_{\text{clean}}(r, p)}$ over all (r, p) , to quantify the distribution shift introduced by imputation. A smaller ratio indicates a smaller relative distribution shift. Although SWD accounts for the multidimensional structure of the data, its value highly depends on the choices of (r, p) .

(5) **Mutual information (MI)**. In information theory, it is a fundamental tool for measuring information dependency between two random variables. For instance, Alexander Strehl et al. [83] used MI to analyze clustering results. Formally, let $X_{\text{imp}}^{(k)}$ and X_{clean} denote the random variables corresponding to $D_{\text{imp}}^{(k)}$ and D_{clean} , $p_{\text{imp}}^{(k)}(x)$ and $p_{\text{clean}}(y)$ denote their respective probability density functions, and $p_{\text{joint}}^{(k)}(x, y)$ represent the joint probability density function, then the MI between $D_{\text{imp}}^{(k)}$ and D_{clean} is

$$I(X_{\text{imp}}^{(k)}; X_{\text{clean}}) = \sum_{x, y} p_{\text{joint}}^{(k)}(x, y) \log \left(\frac{p_{\text{joint}}^{(k)}(x, y)}{p_{\text{imp}}^{(k)}(x) p_{\text{clean}}(y)} \right)$$

which quantifies the extent to which the imputed data retains the information present in the clean data. A higher MI indicates that the imputed data better retains the latent information of the original data. Although MI can quantify the degree of information dependency, it cannot distinguish whether the preserved information is task-relevant or merely noise.

4.3 Performance evaluation

D_{clean} , D_{miss} , and $D_{\text{imp}}^{(k)}$ are evaluated across the tasks in Section 3.4. This multi-task evaluation goes beyond single-task studies, and constitutes, to our knowledge, one of the early attempts to evaluate imputation effects across multiple tasks using a single foundational model (MLP) as a probe. This stage helps establish the pipeline *imputation algorithm analysis* $\rightarrow$ *imputation quality assessment* $\rightarrow$ *performance evaluation*.

However, we identify a fundamental challenge in cross-task performance comparison: the evaluation metrics used for different tasks differ substantially in their nature. Classification uses positive scores (e.g., F1), while regression and time series forecasting typically employ negative-error metrics (e.g., RMSE, MAE). This heterogeneity of metrics makes direct comparisons difficult and obscuring a unified view of imputation impact. To resolve this, we introduce a unified performance loss metric, **PG (Performance Gap)**, a normalized metric that quantifies relative performance loss due to imputation:

$$PG = \frac{\max(0, \pm [f_q(D_{\text{imp}}) - f_q(D_{\text{clean}})])}{f_q(D_{\text{clean}})}$$

where q is a task-specific metric, $f_q(D_{\text{imp}})$ and $f_q(D_{\text{clean}})$ are the metric values of imputed data and clean data, respectively. $f_q(D_{\text{imp}}) - f_q(D_{\text{clean}})$ reflects the performance gap, and the sign is "+" for higher-is-better metrics and "-" for lower-is-better ones. The $\max(0, \cdot)$ ensures only performance degradation is counted, preventing spurious gains from noise or overfitting.

Compared to other metrics, PG offers following advantages.

① **Task-oriented performance deviation measurement.** PG directly compares the performance difference between $D_{\text{imp}}^{(k)}$ and the ideal baseline D_{clean} , rather than evaluating $D_{\text{imp}}^{(k)}$ in isolation, thereby quantifying the impact caused by imputation more accurately. PG incorporates a sign-adaptive mechanism, making it effective for both positive and negative metrics. By truncating at zero, PG retains only the performance loss, ensuring that it always reflects the worst-case performance degradation and preventing interference introduced by factors such as overfitting.

② **Normalization-driven cross-task comparability.** By normalizing against the baseline $f_q(D_{\text{clean}})$, PG transforms absolute performance gaps into relative loss ratios, eliminating differences in units and scale across tasks, which allows performance loss from classification, regression, and time series tasks to be compared fairly within the same numerical space. Moreover, since PG reflects the relative performance decrease proportion, it prevents bias that could arise from variations in D_{clean} itself.

③ **Intuitive and interpretable threshold.** PG directly quantifies imputation-induced harm. When $PG = 0$, it indicates that the model performance under the imputed data is identical to that under the clean data, meaning the imputation causes no harm. The larger the value, the more severe the performance degradation introduced by imputation. More importantly, a performance tolerance threshold can be set directly based on PG without requiring additional data-dependent calibration or unit conversions.

Thus, PG supports convenient and effective cross-task comparison.

4.4 Key factors analysis

Current imputation studies rely heavily on general metrics, which often overlook the varying sensitivity of different tasks to data quality dimensions. To bridge this gap, we deeply identify and formalize the **key factors** that critically influence tasks, as a task-oriented extension of data quality. This step completes the four-stage pathway IIKP-Path and forms the core of our analysis. Based on thorough analysis and literature review of each task, we define the following key factors: (1) **Classification**: label correctness ratio and class discriminability, (2) **Regression**: imputation bias and feature-target correlation, and (3) **Time series forecasting**: recovery fidelity of trend, seasonality, and residuals components. (See Section 5.5 for detailed justification.)

Since the key factors differ among tasks, establishing the pathway involves certain complexity. Therefore, we design a general analysis workflow for downstream tasks, which can be flexibly adapted to the specific needs of each task through appropriate adjustments and extensions. Centered on this workflow, we analyze these key factors through a three-layer progressive study.

Algorithm 1: KEY-FACTORS BOUNDARY FINDER (KFBF)

Input: Datasets $D_{\text{clean}}, D_{\text{imp}}$, a list of given Factor $_i$
 List $_i = [\text{Factor}_{i1}, \text{Factor}_{i2}, \dots, \text{Factor}_{in}]$, the tolerance threshold τ of Factor $_i$, the maximum number maxcounter
Output: A list of the upper and lower bounds of Factor $_j$ List $_j = [(\text{Factor}_{j1u}, \text{Factor}_{j1l}), \dots, (\text{Factor}_{jnu}, \text{Factor}_{jnl})]$

```

1 List $_j \leftarrow []$ 
2 foreach Factor $_{ik} \in \text{List}_i$  do
3   counter $_u \leftarrow 0$ , counter $_l \leftarrow 0$ , Factor $_{jku} \leftarrow -\infty$ ,
   Factor $_{jkl} \leftarrow \infty$ 
4   index  $\leftarrow \text{FindMissing}(D_{\text{clean}}, D_{\text{miss}})$ 
5    $D_{\text{prou}}, D_{\text{prol}} \leftarrow \text{RandomProcessing}(D_{\text{clean}}, \text{index})$ 
6 while true do
7   Factor $'_{ik}, \text{Factor}'_{jk} \leftarrow \text{Calculate}(D_{\text{prou}})$ 
8   if  $|\text{Factor}'_{ik} - \text{Factor}_{ik}| < \tau$  then
9     if Factor $'_{jk} > \text{Factor}_{jku}$  then
10       Factor $_{jku} \leftarrow \text{Factor}'_{jk}$ 
11       counter $_u \leftarrow 0$ 
12     else
13       counter $_u \leftarrow +$ 
14       if counter $_u \geq \text{maxcounter}$  then
15         break;
16   else
17      $D_{\text{prou}} \leftarrow$ 
18       KFCFAA( $D_{\text{prou}}, \text{Factor}_{ik}, \text{Factor}'_{ik}, \text{index}$ ) // Algorithm 2
19     continue;
20    $D_{\text{prou}} \leftarrow \text{KFUAA}(D_{\text{prou}}, \text{index})$  // Algorithm 4
21 while true do
22   Factor $'_{ik}, \text{Factor}'_{jk} \leftarrow \text{Calculate}(D_{\text{prol}})$ 
23   if  $|\text{Factor}'_{ik} - \text{Factor}_{ik}| < \tau$  then
24     if Factor $'_{jk} < \text{Factor}_{jkl}$  then
25       Factor $_{jkl} \leftarrow \text{Factor}'_{jk}$ 
26       counter $_l \leftarrow 0$ 
27     else
28       counter $_l \leftarrow +$ 
29       if counter $_l \geq \text{maxcounter}$  then
30         break;
31   else
32      $D_{\text{prol}} \leftarrow$ 
33       KFCDA( $D_{\text{prol}}, \text{Factor}_{ik}, \text{Factor}'_{ik}, \text{index}$ ) // Algorithm 3
34     continue;
35    $D_{\text{prol}} \leftarrow \text{KFUDA}(D_{\text{prol}}, \text{index})$  // Algorithm 5
36 List $_j.\text{add}((\text{Factor}_{jku}, \text{Factor}_{jkl}))$ 
37 return List $_j$ 

```

(1) **Basic observation of factor performance.** We compute each key factors on D_{clean} and all $D_{\text{imp}}^{(k)}$, and identify which imputation method excel on which factors and explore the connections between algorithmic traits and factor-level outcomes.

(2) **Quantitative assessment of factor impact.** ① **Contribution rate analysis.** To move beyond the limitations of qualitative observation, we introduce two quantification measures. We regress PG on one or more key factors and report R^2 to quantify the variance

in PG explained by them. Originating from variance explanation [15, 16], this quantifies each factor's effect strength on performance, shifting focus from *whether an influence is significant* to *how large the influence is*. ② **Importance ranking**. Employing a controlled-variable method, we perturb one key factor at a time while fixing others, then rank factors by their induced PG changes, effectively isolating individual contributions under multi-factor coupling.

(3) **Exploration of inter-factor interactions**. Prior work (e.g., analysis of [71] on label noise and [38] on class imbalance) focuses on single-factor effects on performance, but overlooks multi-factor interactions, which are more crucial for understanding model behavior. To address this gap, we propose the **Key-Factors Boundary Finder (KFBF)**, a heuristic optimization algorithm designed to characterize the upper and lower bounds of one factor Factor_j while holding another factor Factor_i fixed. By tracing how Factor_j 's boundry shifts as Factor_i varies, it reveals their interplay, and offers a *computable* and *visualizable* tool for understanding multi-factor coupling.

The KFBF algorithm is outlined in Algorithm 1. Its core idea is, for each Factor_{ik} , two independent processes, upper bound exploration and lower bound exploration, are employed. Under the premise of keeping Factor_{ik} within an acceptable range τ , KFBF utilizes the constrained functions KFCAA (Key-Factors Constraint Ascent Adjustment) and KFCDA (Key-Factors Constraint Descent Adjustment), as well as the unconstrained functions KFUA (Key-Factors Unconstraint Ascent Adjustment) and KFUDA (Key-Factors Unconstraint Descent Adjustment), to maximize or minimize Factor_{jk} until convergence, thereby obtaining the upper bound Factor_{jku} and the lower bound Factor_{jkl} . Among them, KFCAA aims to maximize Factor_{jk} while adjusting Factor_{ik} within tolerance τ , but does guarantee monotonic increase in Factor_{jk} . The function KFUA adjusts toward increasing Factor_{jk} without constraints on Factor_{ik} . The functions KFCDA and KFUDA operate analogously for decreasing Factor_{jk} .

The specific adjustment steps of KFBF are, Line 4 calculates the index of missing part, which corresponds to the adjustable part in the upper bound dataset D_{prou} and the lower bound dataset D_{prol} . Lines 7-19 and lines 21-33 calculate the upper bound Factor_{jku} and the lower bound Factor_{jkl} of Factor_{jk} under a fixed Factor_{ik} . Specifically, line 7 and line 21 use the function `Calculate` to obtain the values of Factor_i and Factor_j on the datasets D_{prou} and D_{prol} , respectively. Lines 17, 19 and lines 31, 33 implement the increase or decrease of Factor_{jk} using concrete functions. Line 9 and line 23 provide the conditions for updating the upper and lower bounds, while line 14 and line 28 specify the termination criteria for the loops, i.e., the bounds are not updated after maxcounter adjustment attempts. We illustrate the steps with Example 4.1.

EXAMPLE 4.1. *In the classification task, taking "label correctness ratio" as Factor_i and "class discriminability" as Factor_j , given a set of values for the label accuracy ratio, e.g., $\text{List}_i = [0.1, 0.2, \dots, 0.9]$, the objective is to obtain the upper bound Factor_{jku} and the lower bound Factor_{jkl} of class discriminability, corresponding to each value of label correctness ratio Factor_{ik} , and record the bounds in List_j .*

For each specific label correctness ratio value Factor_{ik} , the process begins by using the given datasets D_{clean} and D_{miss} to identify the missing index via the function `FindMissing`. Then it applies random

label assignment to the missing portion according to the specific label correctness ratio, serving as the initial upper bound dataset D_{prou} and lower bound dataset D_{prol} . Subsequently, the procedure splits into two parts: solving for the upper bound and solving for the lower bound.

① **Upper bound exploration**: *For the current upper bound dataset D_{prou} , the label correctness ratio Factor'_{ik} and the class discriminability Factor'_{jk} are calculated, respectively. It then checks whether the current label correctness ratio Factor'_{ik} falls within the acceptable range of the preset label correctness ratio Factor_{ik} : If acceptable, check whether the current class discriminability Factor'_{jk} is greater than the current upper bound Factor_{jku} . If so, update Factor_{jku} and reset the counter; otherwise, increment the counter. When the counter reaches the preset maximum, the current Factor_{jku} is taken as the final result. However, if not acceptable, the KFCAA function is applied to bring the label correctness ratio back into the acceptable range while attempting to maximize class discriminability. Finally, disregarding the label correctness ratio constraint, the KFUA function is used to further increase class discriminability. The above process iterates until obtaining the final upper bound Factor_{jku} .*

② **Lower bound exploration**: *For the current lower bound dataset D_{prol} , the label correctness ratio Factor'_{ik} and the class discriminability Factor'_{jk} are calculated, respectively. It then checks whether the current label correctness ratio Factor'_{ik} falls within the acceptable range of the preset label correctness ratio Factor_{ik} : If acceptable, check whether the current class discriminability Factor'_{jk} is less than the current lower bound Factor_{jkl} . If so, update Factor_{jkl} and reset the counter; otherwise, increment the counter. When the counter reaches the preset maximum, the current Factor_{jkl} is taken as the final result. However, if not acceptable, the KFCDA function is applied to bring the label accuracy ratio back into the acceptable range while attempting to minimize class discriminability. Finally, disregarding the label correctness ratio constraint, the KFUDA function is used to further decrease class discriminability. The above process iterates until obtaining the final lower bound Factor_{jkl} .*

Finally, add the upper bound Factor_{jku} and the lower bound Factor_{jkl} of class discriminability to List_j . The process proceeds to find the bounds corresponding to the next label correctness ratio value. This continues until all values in List_i have been processed.

Based on the above procedure, the IIKP-Path is completely constructed.

5 EXPERIMENTAL EVALUATION

5.1 Experimental Settings

Datasets. For three downstream tasks, our study employs 18 datasets (see Table 2), where "_" denotes validation sets for the pruning recommendation method, and the rest for IIKP-Path.

Metrics. (1) **Imputation quality metrics**: The five metrics defined in Section 4.2. (2) **General model metrics**: ① PG, as mentioned earlier, our unified cross-task performance metric, computed using F1 Score (classification), MAE (regression), and RMSE (time series forecasting) ② Correlation coefficient $\rho = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y}$ measures linear dependence for random variables X and Y . ③ R^2 , used to quantify factor contribution rates. (3) **Task-specific metrics**: ① RMSE, MAE, and MSE, use distance to measure data error for regression/time series. For a dataset with N samples, based on the true

Table 2: Summary of datasets

Tasks	Datasets	Domain	#row	#col	Source
Classification	Beers	Industry	2410	10	[59]
	Flights	Transportation	2376	6	[74]
	Hospital	Healthcare	1000	19	[74]
	Red Wine	Industry	1599	12	[18]
	Avocado	Agriculture	250	9	[21]
	Glass	Material	214	10	[30]
Regression	Concrete	Material	1030	9	[99]
	CCPP	Energy	9568	5	[84]
	Airfoil	Aviation	1503	6	[9]
	Abalone	Biology	4177	9	[66]
	ParisHP	Economic	10000	17	[65]
	BostonHP	Economic	506	14	[65]
Time series forecasting	ETTh1	Electricity	17420	8	[111]
	ETTm1	Electricity	69680	8	[111]
	Illness	Healthcare	966	8	[96]
	Exchange	Economic	7588	9	[96]
	Weather	Environment	52696	22	[96]
	ETTh2	Electricity	17420	8	[111]

value y_i and the predicted value $\hat{y}_i$ of the target variable in downstream task, the above metrics are: $RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$, $MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$, $MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$. ② Precision, Recall, and F1 Score, used to evaluate the accuracy of classification. Given a dataset with N samples and K classes, for the n_k samples of the k -th class, the number of true positives is TP_k , the number of false positives is FP_k , and the number of false negatives is FN_k . Then, the Precision for the k -th class is $Precision_k = \frac{TP_k}{TP_k + FP_k}$. With the weights $w_k = \frac{n_k}{N}$, the overall Precision across the entire dataset is calculated as $Precision = \sum_{k=1}^K w_k \cdot Precision_k$. Similarly, $Recall_k = \frac{TP_k}{TP_k + FN_k}$, $Recall = \sum_{k=1}^K w_k \cdot Recall_k$. F1 Score is the harmonic mean of Precision and Recall, i.e., $F1\ Score = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$. ③ Label correctness ratio, denoted as $label_{cr} = \frac{Num_C}{Num_T}$, where Num_C and Num_T are correctly imputed and total missing cells. ④ Class discriminability, denoted by $J_d = \frac{d_b}{d_w}$, measures the quality of class boundaries in classification tasks, with inter-class distance d_b and intra-class distance d_w .

Hyperparameters. In experiments, both imputation algorithms and downstream tasks involve tunable hyperparameters. To ensure fairness and reproducibility, we adopt the following strategies: (1) **Downstream tasks.** For each dataset, we perform a random grid search [6] over a broad hyperparameter space, using 5-fold cross-validation to efficiently locate near-optimal configurations. (2) **Imputation algorithms.** We use either the default settings of each algorithm's implementation or typical values widely used in literatures, ensuring all algorithms are compared under reasonable and standard conditions. All code is available at our GitHub.

5.2 Imputation algorithms analysis

Imputation time. All imputation algorithms are applied to the regression and time series datasets in Table 2. For classification datasets, we exclude Mean, Median, and MFI, because Mean and Median are unsuitable for categorical attributes, and MFI fail to finish even one epoch within 24 hours. We record each algorithm's imputation time across all applicable datasets and missing rates. Results are shown in Figure 3a. Based on runtime, algorithms fall into four levels: **Level 1** ($< 1s$): Mean, Median, Mode; **Level 2** ($< 10s$): KNN, HDI, XGBI, SI, MICE, IIM; **Level 3** (minutes): GAIN, MIDAE; **Level 4** (hours): MissFI, MFI. Runtime trends with missing rate vary: KNN and HDI first increase then decrease; MissFI gradually

increases; GAIN gradually decreases. This is because: ① *KNN/HDI*. Initial increase comes from more missing points each requiring neighbor search, and later decline occurs when too few observed samples reduce per-point search cost. ② *GAIN*. Higher missing rates leave the discriminator fewer real samples, weakening its capability and causing earlier convergence. ③ *MissFI*. Data scarcity hurts stability, requiring more iterations to reach steady results, which lengthens runtime.

Validation. We validate certain applicable scenarios of imputation methods in Section 4.1 using regression datasets which are purely numerical and structurally diverse.

(1) **Sample similarity (KNN and HDI).** For each dataset, we calculate the average distance from each missing data point to its nearest neighbor under different missing rates. And then average these distances over all missing data points. The results are shown in Figure 3b, with the vertical axis "Average Distance" scaled to focus on the underlying trend. As missing rate increases, the distance tends to increase, and the downstream performance of KNN and HDI in Section 5.4 also degrade. However, the correspondence is not strict, likely due to internal error propagation or over-/underfitting induced by data.

(2) **Singular value (SI).** To test the low-rank assumption, we perform SVD on D_{clean} and calculate cumulative energy proportion of the first k singular values. That is, $\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^M \sigma_i^2}$. A smaller k together with a larger cumulative energy proportion indicates that data are closer to low-rank. The results are shown in Figure 3c. Energy concentration varies across datasets: CCPP and Abalone show energy concentrated in a few leading singular values, while Concrete, Airfoil, and ParisHP exhibit more dispersed spectra. In Section 5.4, SI performs well on CCPP and Abalone, poorly on Concrete and Airfoil, consistent with their energy distributions. But SI also performs well on ParisHP, which suggests the low-rank assumption may not be the only factor governing SI's effectiveness. Other factors (e.g., noise level, nonlinear relations) may also play a role, which should be identified and analyzed in future work.

(3) **Global variable correlation (MICE).** For each D_{clean} , calculate the correlation coefficient between every pair of variables. The results are shown in Figure 3d, where attributes are represented by index. The strength of global variable correlations varies considerably across datasets. The variables in Concrete and Airfoil show only weak correlations, whereas CCPP and Abalone exhibit a mix of strong and weak correlations. In ParisHP, some variable pairs, such as "price" (index 17 in the figure) and "square meters" (index 1), show extremely strong correlations, while other pairs are essentially uncorrelated. These patterns align with MICE's downstream performance (Section 5.4), i.e., it performs better on CCPP, Abalone, and ParisHP than on Concrete and Airfoil.

5.3 Data Quality Assessment

Results. We evaluate using the five quality metrics from Section 4.2. As these apply only to numerical attributes, we assess regression and time series datasets, and classification datasets (with categorical features) are excluded. The results of KS test, KL divergence, 2-Wasserstein distance, SWD, and MI are shown in Figures 4, Figure 5, Figure 6, Figure 8, and Figure 7, respectively. As missing rates

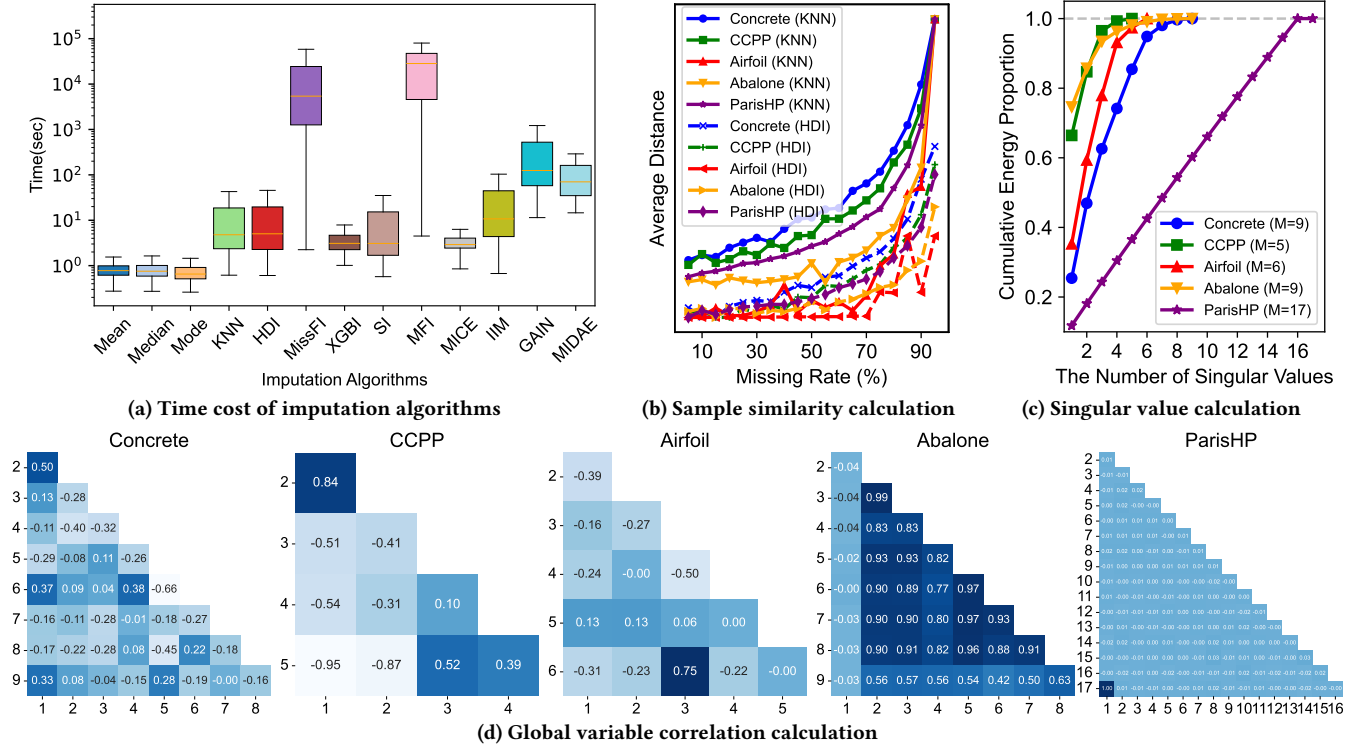


Figure 3: Validation of imputation algorithms.

rise, both distribution fidelity and information dependency preservation decline. Similarity-, tree-, and regression-based methods consistently achieve better data recovery across all metrics, while statistical, compression- and DL-based methods perform slightly worse. This is because, Mean, Median, and Mode produce fixed values and cannot preserve the original distribution, SI and MFI distort structure through matrix factorization, and GAIN and MIDAE suffer from the limitations noted in Section 4.1.

Summary. Finding 1. An algorithm’s imputation quality depends on how well its design mechanisms (e.g., structure utilization, underlying assumptions) align with data characteristics (e.g., missing rate, correlation, complexity, scale) (1) **Structure utilization.** ① *Local structure (KNN, HDI, IIM).* Work well at low missing rates; higher rates increase inter-sample distances and distort local similarities/correlations, degrading performance on all quality metrics. ② *Global relationship (statistical, compression-based and MICE).* Use global data patterns. Statistical methods ignore variable relations and thus perform poorly on distribution metrics; MICE relies on global variable correlations (validated in Section 5.2). Compression-based methods depend on underlying assumptions (discussed below). ③ *Adaptive learning of global and local structures (tree- and DL-based).* Adaptively learn complex structures from global to local, enabling stronger handling of complex data (See Section 4.2). (2) **Underlying assumptions.** MFI assumes linear latent factors; SI assumes low-rank data. When data violate these assumptions, imputation errors arise (see Section 5.2). (3) **Others.** Tree-based methods can handle complex data, explaining its robust performance across quality metrics. DL-based

methods depends on data scale, their performance degrades as the missing rate increases and data shrink.

Finding 2. No universally best imputation algorithm. Relative performance differs clearly across categories, metrics, datasets, and missing rates. This aligns with prior studies [29, 36, 42, 46, 54, 78], and our work further quantifies this variability: ① **No winner across missing rates.** For MI on ETtm1, GAIN is best for $\alpha \leq 10\%$, while SI is best for $\alpha \geq 30\%$. ② **No winner across datasets.** For KL divergence at $\alpha = 30\%$, XGBI is best on Concrete, but HDI on Abalone. ③ **No winner across data quality metrics.** For Abalone ($\alpha = 10\%$), HDI is best on KS test, MissFI on KL divergence, XGBI on 2-Wasserstein distance, IIM on SWD, and SI on MI. Beyond confirmation, our study further uncover two systematic patterns: ① **Between categories.** Statistical, compression-, and DL-based methods consistently show weaker data recovery capability than similarity-, tree-, and regression-based methods across all metrics. ② **Within each category.** Their relative ranking changes within each category with the evaluation metric. *Statistical:* Performance is similar on KS test and MI, but Mode underperforms on 2-Wasserstein and SWD, while Mean/Median are worse on KL divergence for Weather and ParisHP. *Compression-based:* SI almost always outperforms MFI across all datasets on KS test, 2-Wasserstein distance, SWD, and MI, but the reverse holds for KL divergence (e.g., on Abalone and ParisHP). *DL-based:* MIDAE shows competitive MI on time series but poor distribution recovery, while GAIN performs worse overall. These trends arise from differences in both metric formulations and algorithmic principles (Section 4.2). All findings

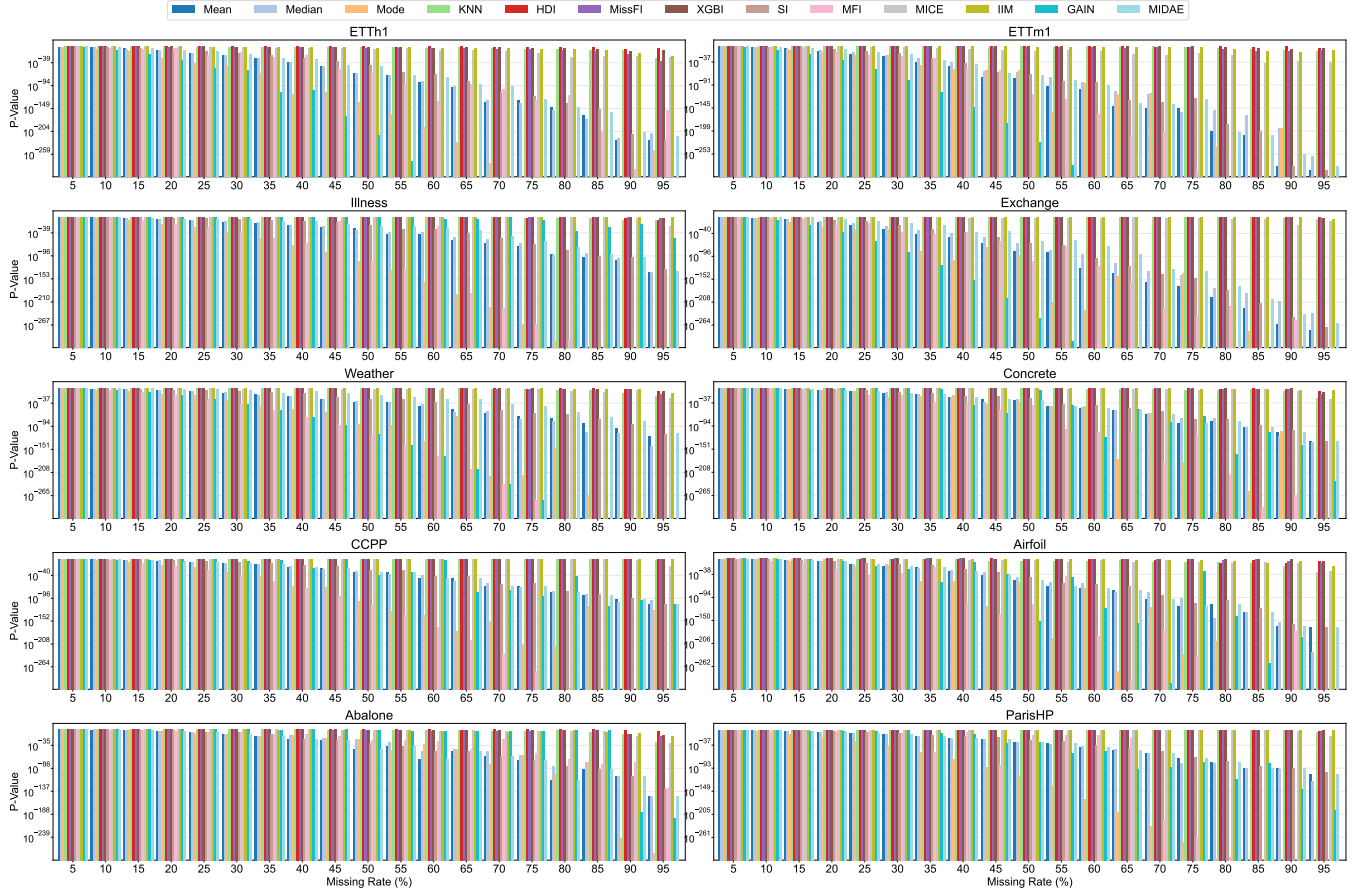


Figure 4: The results of KS test.

hold consistently across missing rates, datasets, and evaluation criteria.

5.4 Performance Evaluation

Results. Figures 9 presents the performance results. When the missing rate increases, regression and time series forecasting suffer greater performance degradation than classification. In regression and time series forecasting, algorithms differ considerably. Some algorithms (e.g., MFI) exhibit large fluctuations, while others (e.g., MICE, XGBl) vary less. In contrast, for classification, most imputation algorithms demonstrate moderate changes. Finally, we observe $PG = 0$ in some cases. Since real-world datasets inevitably contain noise, the smoothing inherent in imputation may incidentally correct certain issues.

Summary. Finding 3. The impact of missingness and imputation on model/tasks: both common and task-specific patterns. (1) **Common patterns.** The performance decline with rising missing rates confirm [61, 62], which stems from MLP’s structure. In forward propagation, an error introduced by missingness or imputation propagates layerwise. Denoting it as $\hat{y}_{l-1}^{(i)} = y_{l-1}^{(i)} + \theta$, its impact is $\hat{u}_l^{(j)} = \sum_{i \in \text{layer}_{l-1}} w_l^{(ji)} (y_{l-1}^{(i)} + \theta) + b_l^{(j)}$, $\hat{y}_l^{(j)} = f(\hat{u}_l^{(j)})$. In backward propagation, the error cause bias in gradient $\delta_l^{(j)} = \frac{\partial \mathcal{L}}{\partial \hat{u}_l^{(j)}}$,

which affects parameter updates and ultimately the output. (2) **Task-specific patterns.** The distinct task configurations (Section 3.4) explain the performance differences: Classification exhibits greater tolerance to errors from missingness or imputation and less variation across imputation algorithms than do regression and time series forecasting. ① *Classification.* For two dominant categories i and j in the logits $\mathbf{z}$, their difference is $\Delta z = z_i - z_j$. After applying the softmax, the final prediction is category i if $\Delta z > 0$ ($\hat{p}_i > \hat{p}_j$), otherwise category j . When a sample lies near the decision boundary (i.e., $\Delta z \approx 0$), the error can easily change the sign of Δz , causing a probability jump. Conversely, when $|\Delta z| \rightarrow \infty$ ($\hat{p}_i \rightarrow 1$ or $\hat{p}_j \rightarrow 1$), even if an error exists, the sign of Δz tends to remain unchanged. Therefore, classification tasks are sensitive to boundary samples but robust to samples deep within a class. Under high missing rates, all imputation algorithms will disturb several boundary samples, and the distinction between class boundaries and interiors becomes blurred since the number of known samples is small. Consequently, sign changes in Δz will occur more frequently, thus they all exhibit a certain degree of changes. ② *Regression.* For an error $\theta \in \mathbb{R}^d$, the change in output is $\Delta \hat{y} = f(x + \theta) - f(x)$, where $f(\cdot)$ in practice is often Lipschitz continuous or at least locally Lipschitz. Thus, there exists a constant $C \in \mathbb{R}$ such that $0 \leq \Delta \hat{y} \leq C \|\theta\|$. Compared to boundary samples in classification, the change in $\Delta \hat{y}$ is gradual. However, compared to samples deep within a class, the change in

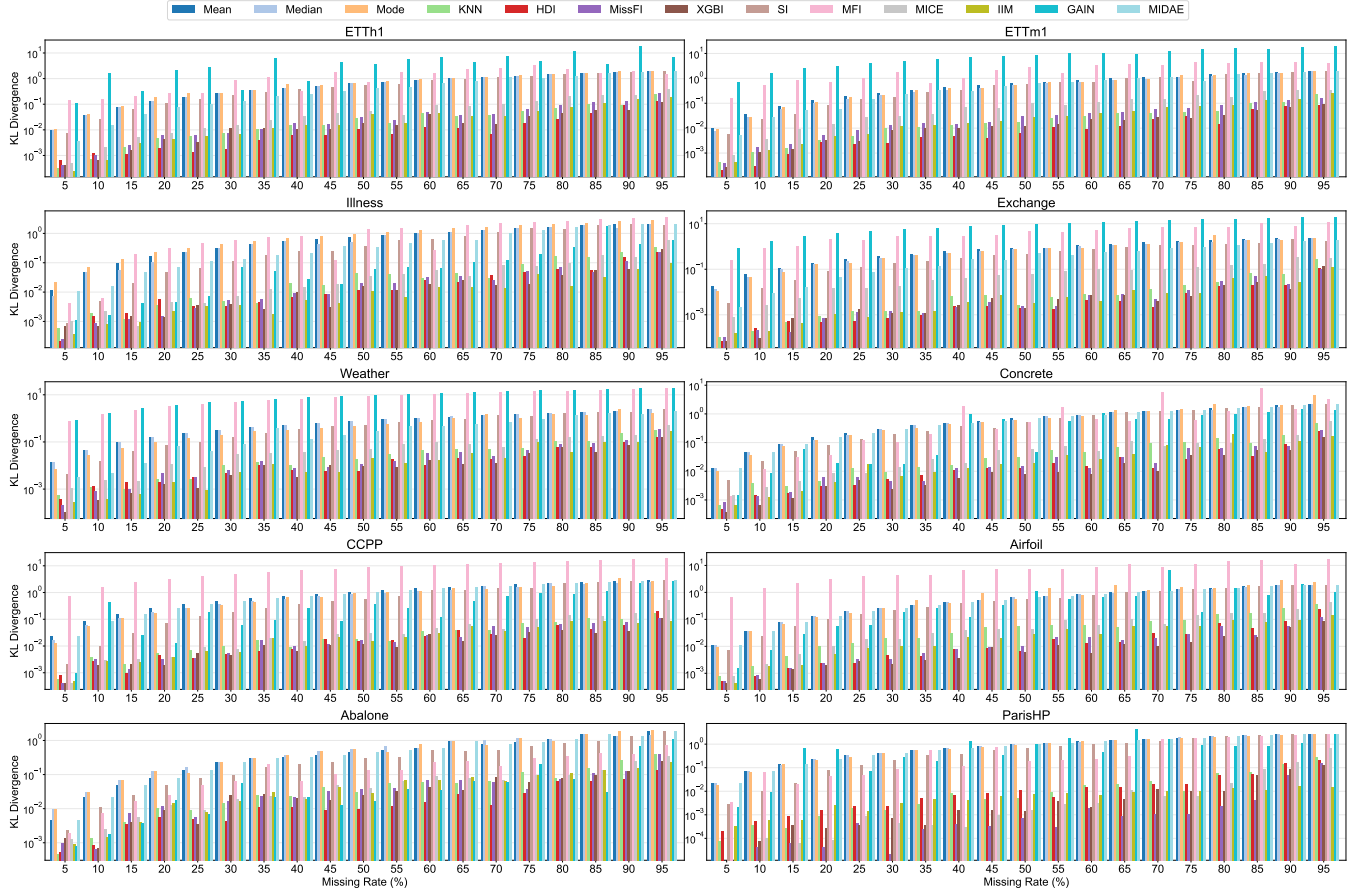


Figure 5: The results of KL divergence.

$\Delta\hat{y}$ cannot be ignored and will be reflected in the final result. The same principle applies to *time series forecasting*. Because the error θ is directly incorporated into $\Delta\hat{y}$, different imputation algorithms introduce different errors, leading to significant differences in output, especially as the missing rate increases and errors accumulate.

Finding 4. The task performance is generally positively correlated with imputation quality, but not in a one-to-one manner. Figure 10 plots task performance against imputation quality across all configurations (each contains 10 datasets $\times$ 13 algorithms $\times$ 19 missing rates), which sufficiently supports our findings. The blue trend line is fitted based on all scatter points and generally reflects the overall trend of task performance as the imputation quality changes. From the overall trend, better imputation quality corresponds to better performance. Imputation algorithms that achieve better imputation quality in Section 5.3 also perform better in downstream tasks. However, studies [46, 78] have suggested that general imputation quality are not reliable predictors of downstream performance. Our study observed similar phenomena. In Figure 10, very different quality values can correspond to similar PG values, and similar quality values can map to very different PG values. This occurs because each imputation quality metric has inherent limitations, as discussed in Section 4.2. On one hand, our study employs all of them to assess from various perspectives. On the other hand, it indicates that, merely focusing on general

quality metrics is insufficient. It is necessary to extend data quality dimensions for specific tasks, i.e., the key factors discussed below.

5.5 Key Factors Analysis

In this section, we leverage the *task-specific key factors* to conduct a more in-depth analysis. The workflow is illustrated in Figure 11. The label correctness ratio, imputation bias, and the correlation coefficients of each component are also used to measure the sample discrepancy in Section 4.2.

5.5.1 Classification. Studies [8, 69] have demonstrated that the label correctness has a significant impact on classification performance. [28] points out that the lower bound of error rates on a given dataset (the Bayes error) is closely related to the inter-class overlap (i.e., class discriminability). Therefore, we identify **label correctness ratio** and **class discriminability** as the key factors for classification. The workflow is illustrated in Figure 11(a).

Basic observation. Figure 12a and Figure 12b respectively show the label correctness ratio and class discriminability of $D_{\text{imp}}^{(k)}$. Both decline as the missing rate increases, but they vary considerably across datasets. *Label correctness ratio*: Similarity-based methods and MissFI outperform other methods on most datasets. On Beers, however, all methods perform poorly, with HDI being the top performer. *Class discriminability*: On the Flights, Hospital, and Avocado

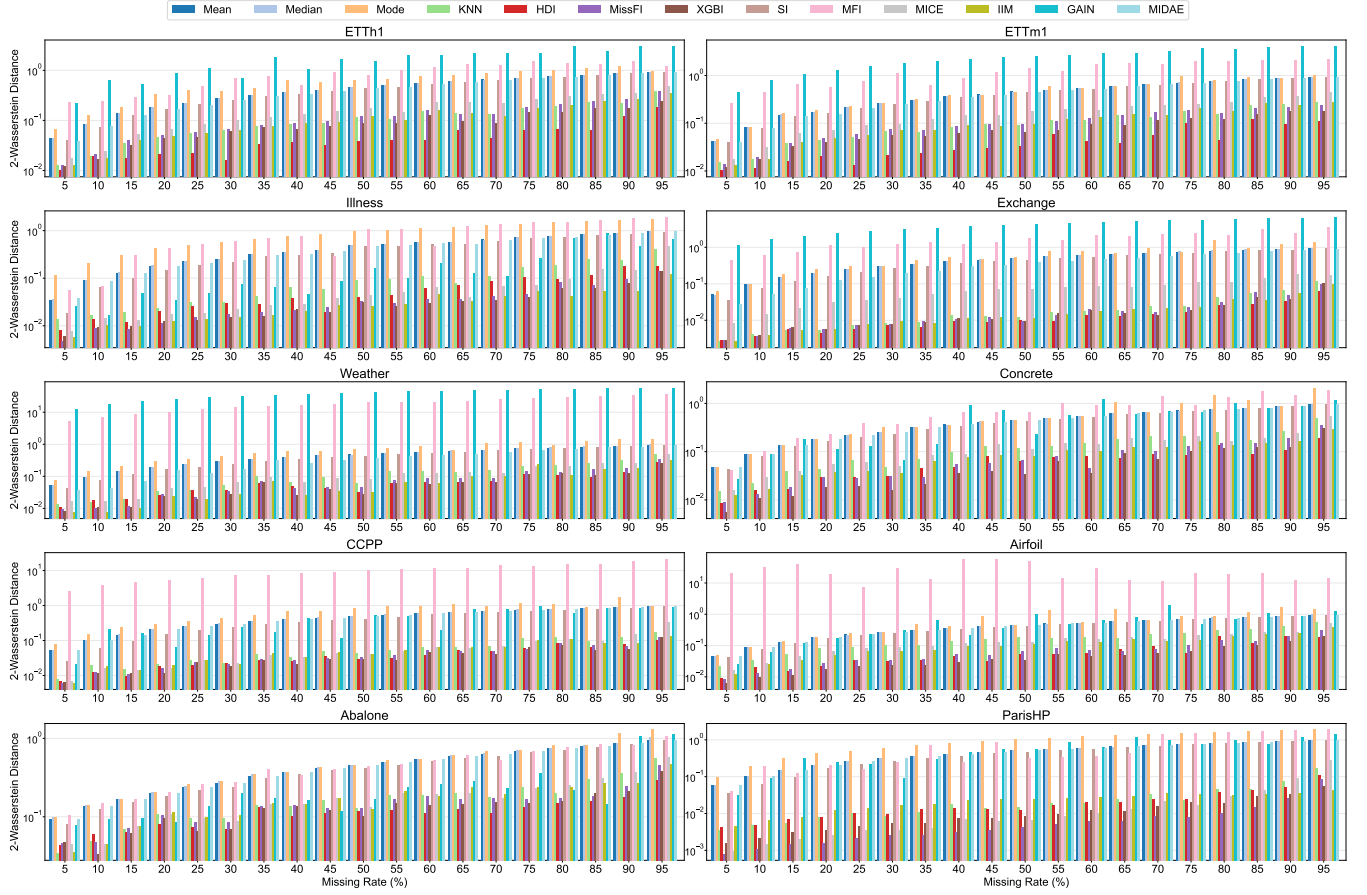


Figure 6: The results of 2-Wasserstein distance.

Table 3: Contribution rate analysis of classification tasks

Independent variables	Beers	Flights	Hospital	Red Wine	Avocado
Label correctness ratio	0.081	0.269	0.321	0.363	0.409
Class discriminability	0.827	0.628	0.743	0.461	0.558
Both of them	0.833	0.658	0.781	0.529	0.558

datasets, KNN, HDI, and MissFI achieve the best class discriminability. On Beers, results are mixed with no clear leader, while on Red Wine, all methods perform poorly, with MICE slightly ahead.

Quantitative assessment. We perform the contribution rate analysis, the results are presented in Table 3. When using both label correctness ratio and class discriminability as independent variables, R^2 all exceed 0.5, indicating certain interpretability. Moreover, R^2 for class discriminability are significantly higher than those for the label correctness ratio, suggesting that class discriminability contributes more.

Exploration. We use KFBF in Section 4.4 to calculate the upper and lower bounds of class discriminability under given label correctness ratios. We implement KFCAA (Algorithm 2), KFCD (Algorithm 3), KFUA (Algorithm 4), and KFUDA (Algorithm 5) for KFBF.

(1) **KFCAA and KFCD** perform symmetrical operations: Line 1 calculates the number of labels to be adjusted, denoted as num . If the label correctness ratio is too large, calculate the distance from each correct data point to its proper category center (line 3). For the upper bound, sort these distances in descending order; for the lower

bound, sort them in ascending order, and store them in $List_{dc2c}$ (line 4). Take the first num data points and modify their labels to the nearest (upper bound) or farthest (lower bound) incorrect category (line 5). If the label correctness ratio is too small, calculate the distance from each incorrect data point to its proper category center (line 7). For the upper bound, sort these distances in ascending order; for the lower bound, sort them in descending order, and store them in $List_{dw2c}$ (line 8). Take the first num data points and modify their labels to the correct category (line 9).

(2) Line 1 of **KFUA** sets the maximum number of adjustments per iteration $maxadjust$. Line 2 **FindDifference** retrieves the indices $index'$ of all data points whose current label differs from the nearest category label. Lines 4 and 5 calculate the corresponding distances and sort the distances in ascending order. To prevent the iterative process from oscillating repeatedly between KFCAA and KFUA and falling into a local optimum, each adjustment must include at least one data point whose current label is correct and one incorrect. Therefore, num is at least two and at most the smaller of the adjustable number $len(index')$ and $maxadjust$. Finally, based on the two selected data points, choose the first $num - 2$ non-repeating data points of $List_{dmin}$, i.e., adjust the total of num labels, and modify their labels to the nearest category.

(3) Line 1 of **KFUDA** is the same. Line 2 **FindSame** retrieves the index of all data points whose labels match the nearest category.

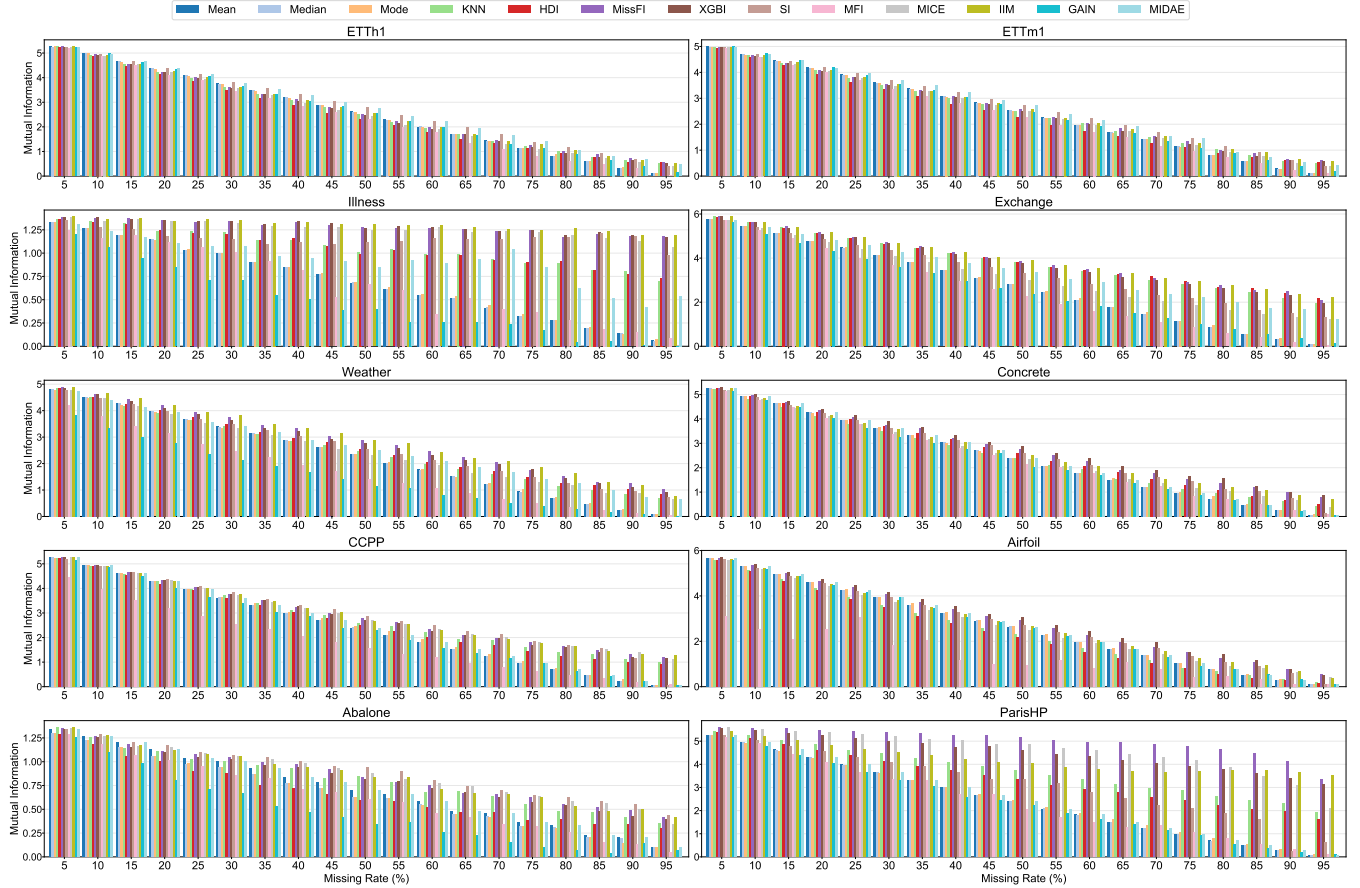


Figure 7: The results of MI.

The design of num in line 3 is the same as above. Lines 4 and 5 calculate the corresponding distances and sort the distances in descending order. Finally, based on the two already selected data points (one correctly labeled and one incorrectly labeled), choose the top $num - 2$ non-repeating data points from $List_{d_{max}}$ and change their labels to the farthest non-identical class.

Based on the above algorithms, the results in Figure 13 reveal that class discriminability spans a wide range of values. We further test downstream performance on five random datasets (generated by five independent runs of line 5 in KFBB) and two datasets at the bounds. Figure 14 presents the results. On the random datasets, performance improves as the label correctness ratio increases. However, the datasets at the bounds perform even worse than the random ones. This observation aligns with [71], which states that classifiers are generally more robust to *uniform noise* (our random datasets) than to *feature-dependent noise* (the upper/lower bounds datasets).

Summary. Finding 5. Imputation methods perform differently in label correctness ratio and class discriminability, which is related to the datasets. Data in classification tasks typically exhibit clustering characteristics, enabling similarity-based methods working effectively. Tree-based methods can handle complex data as discussed in Section 4.1 but differ in stopping criteria: MissFI stops when error first increases, whereas XGBI continues until error falls below the threshold. This makes XGBI prone to

Algorithm 2: KEY-FACTORS CONSTRAINT ASCENT ADJUSTMENT (KFAA)

Input: D_{prou} , $Factor_{ik}$, $Factor'_{ik}$, index
Output: Adjusted D_{prou}

```

1 num  $\leftarrow$  len(index)  $\cdot$   $|Factor'_{ik} - Factor_{ik}|$ 
2 if  $Factor'_{ik} - Factor_{ik} > \tau$  then
3   Listdc2c  $\leftarrow$  Distance_c2c( $D_{prou}$ , index)
4   Listdc2c  $\leftarrow$  Sort(Listdc2c, descent)
5    $D_{prou} \leftarrow$  Relabel_c2w( $D_{prou}$ , Listdc2c, num)
6 else
7   Listdw2c  $\leftarrow$  Distance_w2c( $D_{prou}$ , index)
8   Listdw2c  $\leftarrow$  Sort(Listdw2c, ascent)
9    $D_{prou} \leftarrow$  Relabel_w2c( $D_{prou}$ , Listdw2c, num)
10 return  $D_{prou}$ 

```

overfitting, whereas MissFI achieves higher label correctness ratio. KNN, HDI, and MissFI excel at recovering labels, leading to higher class discriminability on datasets with clear class distinctions (Flights, Hospital, and Avocado). On Beers, moderately high discriminability despite low correctness is attributed to misleading imputation of erroneous labels, which artificially inflates class discriminability. On Red Wine, extremely low discriminability despite moderate label correctness ratio, with MICE achieves a slightly

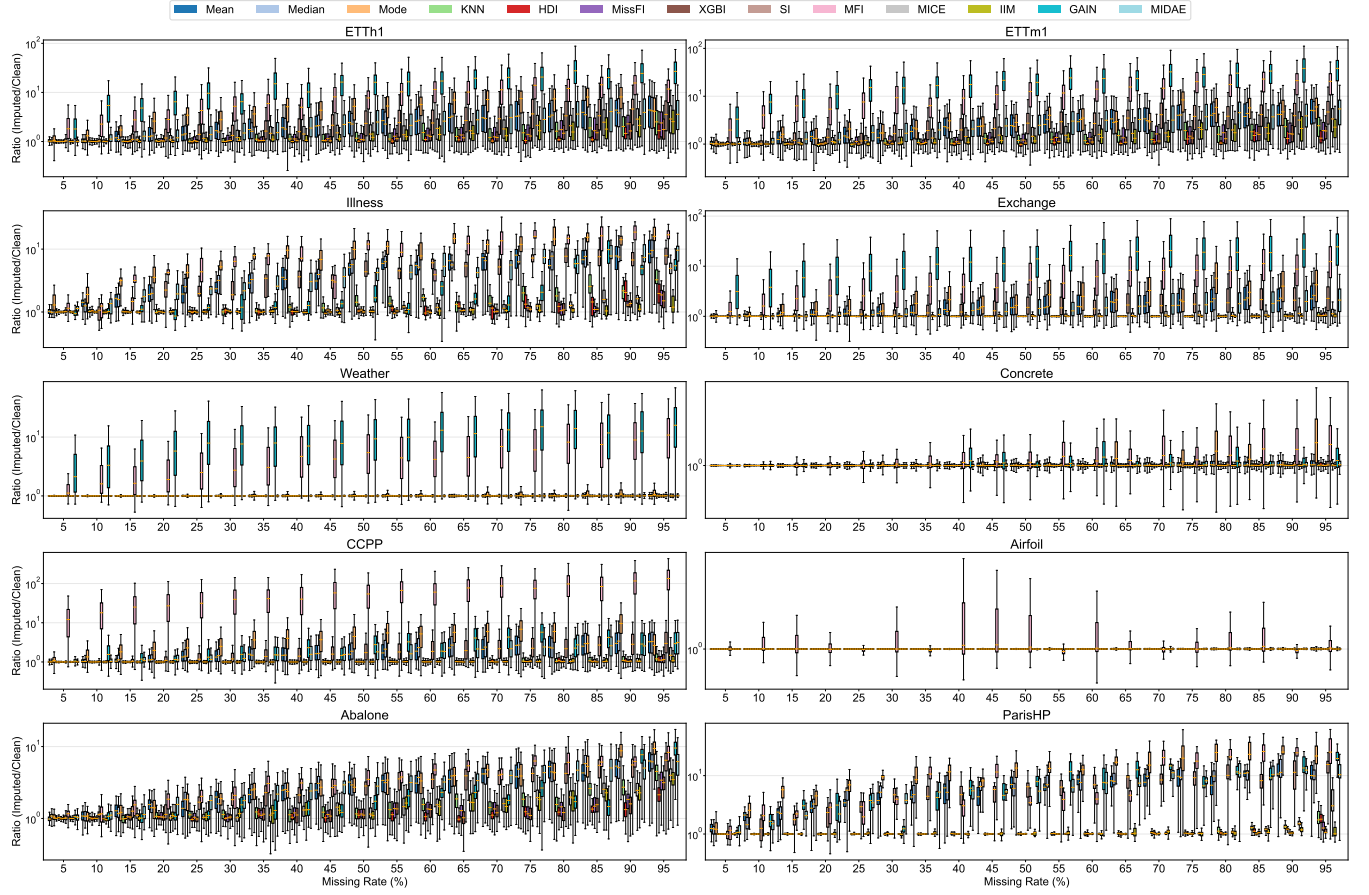
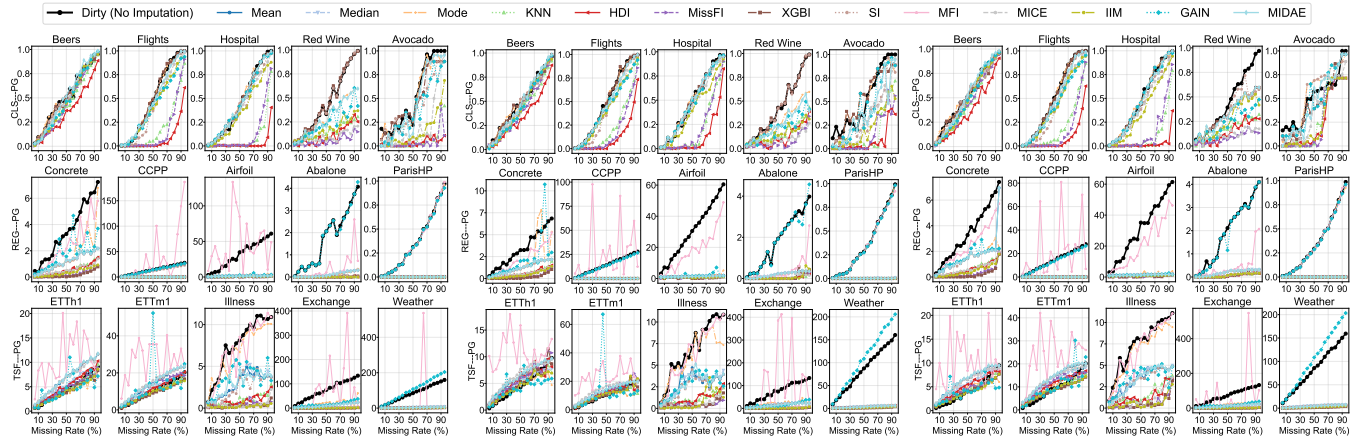


Figure 8: The results of SWD.



(a) Missing Completely At Random (MCAR)

(b) Missing At Random (MAR)

(c) Missing Not At Random (MNAR)

Figure 9: Downstream performance of MLP's three tasks.

higher score since it may amplifies spurious variable dependencies. This also confirms *Finding 6*, that is, the label correctness ratio does not determine the specific value of class discriminability.

Finding 6. Label correctness ratio determines the upper/lower bounds of class discriminability, but not its specific value, as the latter can vary significantly with changes in label distribution. Consequently, relying solely on label correctness ratio

is insufficient; class discriminability must be considered. The label correctness ratio constrains the number of correct and incorrect labels, thereby determining the upper/lower bounds of class discriminability. However, for a given label correctness ratio, different distributions of incorrectly imputed labels can lead to vastly different class discriminability values. Since that value varies widely

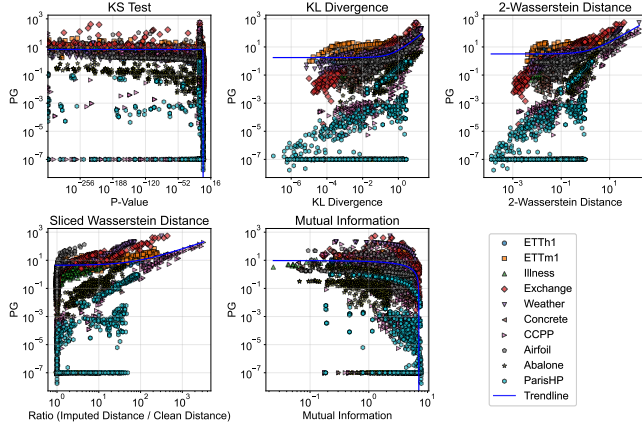


Figure 10: The distribution of quality and performance.

Algorithm 3: KEY-FACTORS CONSTRAINT DESCENT ADJUSTMENT (KFCDA)

Input: D_{prol} , Factor_{ik} , Factor'_{ik} , index
Output: Adjusted D_{prol}

- 1 $\text{num} \leftarrow \text{len}(\text{index}) \cdot |\text{Factor}'_{ik} - \text{Factor}_{ik}|$
- 2 **if** $\text{Factor}'_{ik} - \text{Factor}_{ik} > \tau$ **then**
- 3 $\text{List}_{\text{dc2c}} \leftarrow \text{Distance_c2c}(D_{\text{prol}}, \text{index})$
- 4 $\text{List}_{\text{dc2c}} \leftarrow \text{Sort}(\text{List}_{\text{dc2c}}, \text{ascend})$
- 5 $D_{\text{prol}} \leftarrow \text{Relabel_c2w}(D_{\text{prol}}, \text{List}_{\text{dc2c}}, \text{num})$
- 6 **else**
- 7 $\text{List}_{\text{dw2c}} \leftarrow \text{Distance_w2c}(D_{\text{prol}}, \text{index})$
- 8 $\text{List}_{\text{dw2c}} \leftarrow \text{Sort}(\text{List}_{\text{dw2c}}, \text{descent})$
- 9 $D_{\text{prol}} \leftarrow \text{Relabel_w2c}(D_{\text{prol}}, \text{List}_{\text{dw2c}}, \text{num})$
- 10 **return** D_{prol}

Algorithm 4: KEY-FACTORS UNCONSTRAINT ASCENT ADJUSTMENT (KFUAA)

Input: D_{prou} , index
Output: Adjusted D_{prou}

- 1 Set maxadjust
- 2 $\text{index}' \leftarrow \text{FindDifference}(D_{\text{prou}}, \text{index})$
- 3 $\text{num} \leftarrow \text{Random}(2, \min(\text{maxadjust}, \text{len}(\text{index}')))$
- 4 $\text{List}_{\text{dmin}} \leftarrow \text{Distance_min}(D_{\text{prou}}, \text{index}')$
- 5 $\text{List}_{\text{dmin}} \leftarrow \text{Sort}(\text{List}_{\text{dmin}}, \text{ascend})$
- 6 $D_{\text{prou}} \leftarrow \text{Relabel_min}(D_{\text{prou}}, \text{List}_{\text{dmin}}, \text{num})$
- 7 **return** D_{prou}

across its possible range, it cannot be determined. Specifically, under the same label correctness ratio, it is obvious that mislabeling near class centers has a far greater negative impact on class discriminability than mislabeling near class boundaries. Therefore, the evaluation should extend beyond the label correctness ratio and pay more attention on class discriminability.

Finding 7. The performance of classification tasks is weakly correlated with label correctness ratio and strongly correlated with class discriminability. The class discriminability is dominant. According to the contribution rate analysis results, Both label correctness ratio and class discriminability are potential influencing

Algorithm 5: KEY-FACTORS UNCONSTRAINT DESCENT ADJUSTMENT (KFUDA)

Input: D_{prol} , index
Output: Adjusted D_{prol}

- 1 Set maxadjust
- 2 $\text{index}' \leftarrow \text{FindSame}(D_{\text{prol}}, \text{index})$
- 3 $\text{num} \leftarrow \text{Random}(2, \min(\text{maxadjust}, \text{len}(\text{index}')))$
- 4 $\text{List}_{\text{dmax}} \leftarrow \text{Distance_max}(D_{\text{prol}}, \text{index}')$
- 5 $\text{List}_{\text{dmax}} \leftarrow \text{Sort}(\text{List}_{\text{dmax}}, \text{descent})$
- 6 $D_{\text{prol}} \leftarrow \text{Relabel_max}(D_{\text{prol}}, \text{List}_{\text{dmax}}, \text{num})$
- 7 **return** D_{prol}

factors for classification performance. ① The classification task employs aforementioned cross-entropy loss. When a label incorrectly changed from class i to class j , the loss $\mathcal{L}_{\text{cls}}$ shifts from $-\log \hat{p}_i$ to $-\log \hat{p}_j$. For the backpropagation $\delta_l^{(j)} = \frac{\partial \mathcal{L}}{\partial u_l^{(j)}}$, this introduces error in the gradient direction. Therefore, the label correctness ratio determines whether the MLP can update its parameters along the correct gradient direction. ② High class discriminability implies the outputs $y_l^{(j)}$ at each layer of MLP are intra-class cohesive and inter-class separable. Considering the dominant class i , after Softmax, $\hat{p}_i \rightarrow 1$ and $\mathcal{L}_{\text{cls}} = -\log \hat{p}_i \rightarrow 0$, enabling rapid convergence. In contrast, low class discriminability results in probabilities across K classes being similar, resulting in $\mathcal{L}_{\text{cls}} \approx \log K$ and, consequently, a large gradient. Moreover, similar features corresponding to different labels also cause unstable gradient directions. Thus, class discriminability affects the learning difficulty of MLP. Finally, although imputation may introduce some confusion in the data, it is less damaging compared to those feature-dependent noise, since initial missingness is random. In this context, the learning difficulty of MLP becomes dominant.

5.5.2 Regression. Imputation bias directly reflects the correctness of the knowledge acquired by MLP during regression. Since regression mainly predicts the target variable through feature variables, the **Feature-target correlation** is crucial. It is also the core idea of LARS algorithm [25], which is widely used for regression analysis. Both of them serve as key factors of regression tasks. The workflow is illustrated in Figure 11(b).

Basic observation. The results of imputation bias are shown in Figure 15. For each dataset and under each missing rate, we ranked the imputation algorithms in ascending order by their imputation bias in terms of RMSE/MSE and MAE, respectively. All results were then aggregated for statistical analysis, meaning each imputation algorithm corresponds to results across 5 datasets $\times$ 19 missing rates $\times$ 2 different distance metrics. The results of feature-target correlation are shown in Figure 16, where we calculated the average correlation between each feature and the target variable on $D_{\text{imp}}^{(k)}$. We find that the imputation algorithms with smaller imputation bias largely align with those showing higher feature-target correlations (absolute value). Regression- and tree-based methods perform the best, while similarity-based methods also perform well.

Contribution analysis. Since we observe that the rankings of different imputation algorithms are similar regardless of whether RMSE/MSE or MAE is used to measure imputation bias, we use

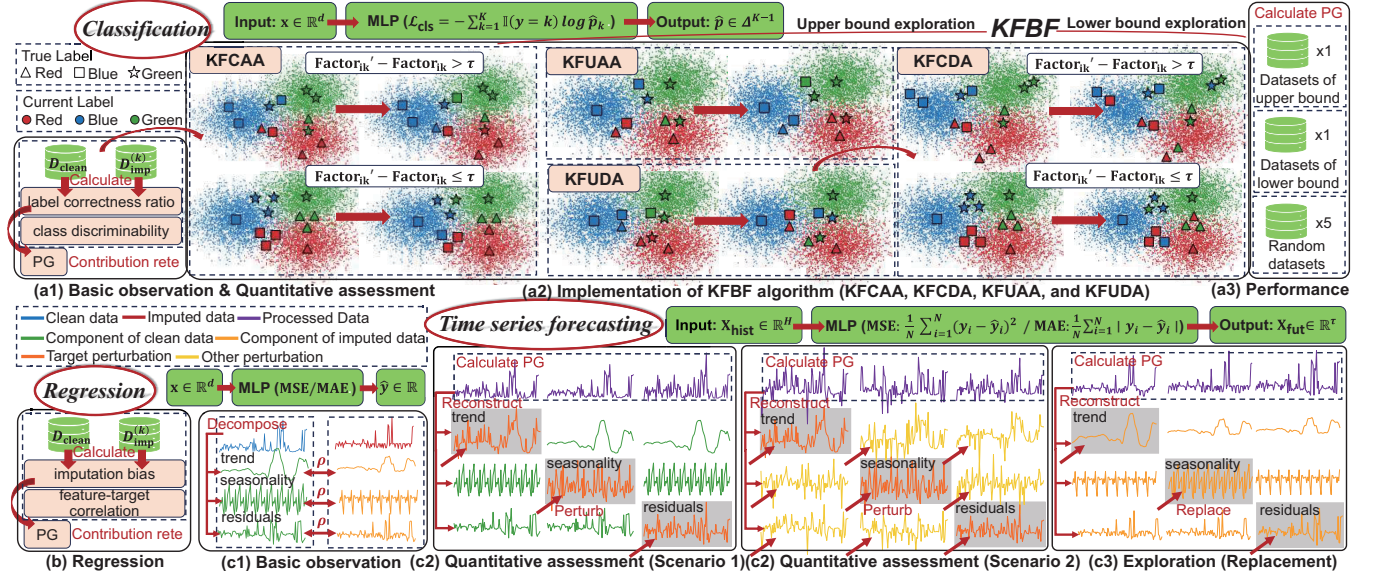


Figure 11: The overview of downstream tasks.

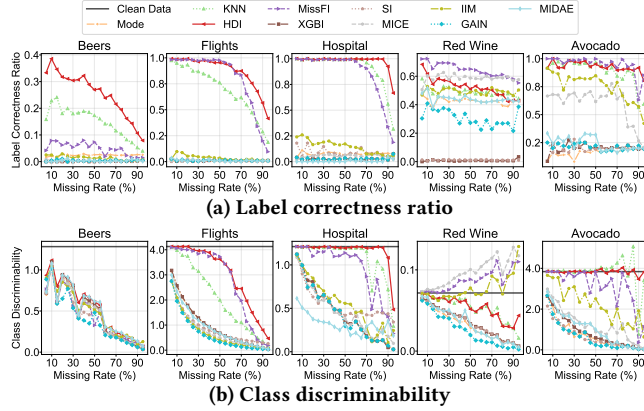


Figure 12: Factors' performance in classification task.

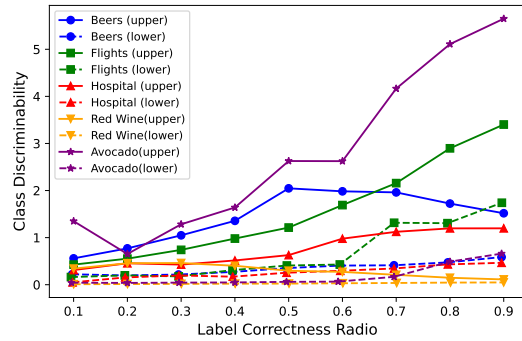


Figure 13: Upper and lower bounds of class discriminability.

Table 4: Contribution rate analysis of regression tasks

Independent variables	Concrete	CCPP	Airfoil	Abalone	ParisHP
Imputation bias	0.938	0.974	0.864	0.606	0.630
Feature-target correlation	0.773	0.212	0.263	0.347	0.521
Both of them	0.942	0.979	0.864	0.607	0.637

RMSE here to represent imputation bias. We perform the contribution rate analysis, the results are presented in Table 4. The R^2

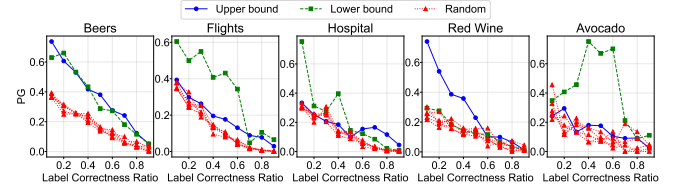


Figure 14: Downstream performance of seven datasets in classification.

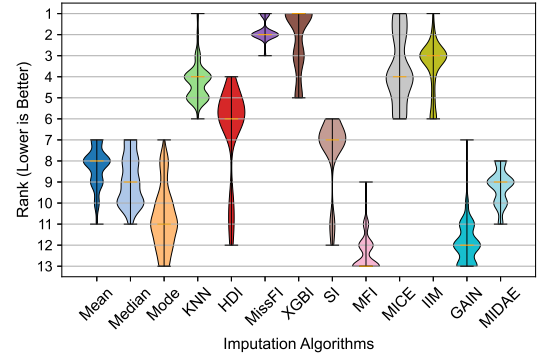


Figure 15: The results of imputation bias in regression.

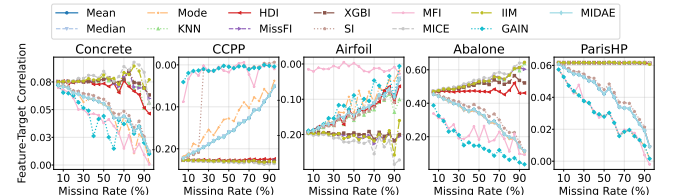


Figure 16: Feature-target correlation in regression.

values for both of imputation bias and feature-target correlation are consistently high, indicating interpretability. Meanwhile, R^2 for each of them varies across datasets. On Concrete and Abalone, both exhibit high. On CCPP and Airfoil, that of imputation bias is very

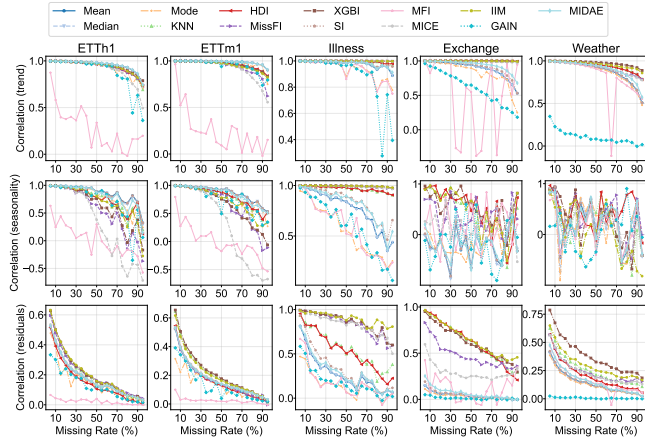


Figure 17: Correlation coefficient in time series forecasting.

high, while that of feature-target correlation is relatively low. On ParisHP, both are moderate.

Summary. Finding 8. Imputation algorithms show consistency between imputation bias and feature-target correlation. We find that across all datasets, imputed data with low imputation bias consistently exhibits high feature-target correlation. Since all imputation methods aim to learn from observation to impute missingness, if it is well-learned (i.e., low imputation bias), it can more effectively restore the variable correlation. *Regression- and tree-based methods* perform best. Regression-based imputation essentially establishes a mapping relationship between observed and missing variables. Hence, the imputation process is formalized as the explicit modeling and iterative optimization of variable dependencies. Therefore, low imputation bias implies better recovery of dependency relationships. Tree-based methods aim to predict the target missing variables based on existing features in each iteration. Regardless of whether the splitting criterion is based on Gini index, variance, or gain, the goal is to make the nodes after splitting purer. As a result, features with high discriminative and predictive power are more likely to be selected, enabling low imputation bias while simultaneously learning and restoring feature-target correlations. *Similarity-based methods* performs well. Similarity-based imputation does not directly model variable dependencies. However, its underlying principle, *similar features correspond to similar targets*, implicitly leverages the correlation between feature variables and the target variable.

Finding 9. The performance of MLP-based regression tasks is influenced by imputation bias and feature-target correlation, with varying impact across datasets. Imputation bias is the dominant factor. Imputation bias and feature-target correlation are potential factors influencing regression task performance. ① *Imputation bias*. Its impact is similar to the analysis in Finding 3. ② *Feature-Target correlation*. When certain feature-target correlations are high, features x and the target y exhibit consistent variation. During iterative training, greater weight $w_l^{(ji)}$ is assigned to these relevant features, allowing $y_i - \hat{y}_i$ to quickly converge to a small value. Consequently, in the m -th layer $\frac{\partial \mathcal{L}}{\partial y_l^{(j)}} \propto (y_i - \hat{y}_i)$, the magnitude of $\delta_l^{(j)}$ also becomes smaller and more stable. Conversely,

when feature-target correlations are low, weights are more dispersed, resulting in larger values of $y_i - \hat{y}_i$ and $\delta_l^{(j)}$, which leads to larger and unstable gradients. According to the contribution rate analysis, in most cases, imputation bias shows stronger correlation.

5.5.3 Time series forecasting. [5, 13, 92] have introduced numerous decomposition-based time series forecasting models, and decomposition has been a crucial direction in time series. Time series is considered to consist of four components: trend (T), seasonality (S), cyclical (C), and irregular (I) [14, 19]. Seasonal decomposition is a commonly used method. It breaks down a time series into three components: trend, seasonality, and residuals (which include both C and I). The additive model is Time Series = Trend + Seasonality + Residuals. Figure 11(c) presents the workflow.

Basic observation. Apply seasonal decomposition to time series, yielding three components. Calculate the correlation coefficients between $D_{\text{imp}}^{(k)}$ and D_{clean} for each component. The results are presented in Figure 17. As missing rate increases, the correlation coefficients mostly decline. Across all datasets, similarity-, tree- and regression-based methods consistently show relatively high correlation coefficients across all components, with IIM and MissFI particularly outstanding.

Importance analysis. To further investigate the impact of three components on downstream performance, we test in two scenarios. Let the target component be c_T and the others be c_{o1}, c_{o2} . The importance of c_T is measured by importance ranking in Section 4.4.

Scenario 1. Measure performance gain when improving c_T while c_{o1}, c_{o2} stay clean. We inject random perturbations into c_T of D_{clean} to produce D_{pro} such that the correlation ρ between the c_T of D_{pro} and D_{clean} spans $\{0.7, 0.72, \dots, 0.98\}$, then evaluate downstream performance.

Scenario 2. Measure performance gain when improving c_T while c_{o1}, c_{o2} are dirty. We first perturb c_{o1}, c_{o2} so their correlations with D_{clean} are fixed at 0.5, then perturb c_T to the same ρ range as in Scenario 1 and evaluate performance.

Table 5 presents the results. When the correlation coefficient is 0.7, the performance is most significantly impacted when c_T is trend, followed by residuals, and finally seasonality. As the correlation coefficient increases from 0.7 to 0.98, the performance does not show a consistent improvement. Compared to Scenario 2, more performance improvement cases are observed in Scenario 1.

Exploration. Based on the results in Section 5.4, we select imputation algorithms that performed poorly under $\alpha \in \{50\%, 70\%, 90\%\}$, i.e., Mean, Median, Mode, SI, MFI, GAIN, and MIDAE, whose imputed data to be replaced. We identify the imputed data that achieves the highest correlation coefficient on c_T at each missing rate in Basic Observation, and use it to replace, while keeping others unchanged, thereby generating D_{pro} . Figure 18 present the percentage PG change from pre- to post-replacement. We observe that replacing components does not always lead to performance improvement, e.g., replacing the seasonality of SI in Weather when $\alpha = 70\%$, or replacing the residuals of Mean in ETTh1 when $\alpha = 90\%$. When downstream performance is improved, replacing "trend" yields the greatest gain (the solid-colored bars are often at the bottom), followed by residuals, then seasonality.

Table 5: Downstream performance of different scenarios in time series forecasting

Datasets		ETTh1						ETTm1						Illness						Exchange						Weather					
c_r		T		S		R		T		S		R		T		S		R		T		S		R		T		S		R	
ρ		0.7	0.98	0.7	0.98	0.7	0.98	0.7	0.98	0.7	0.98	0.7	0.98	0.7	0.98	0.7	0.98	0.7	0.98	0.7	0.98	0.7	0.98	0.7	0.98	0.7	0.98	0.7	0.98	0.7	0.98
MLP	Scenario 1	3.09	0.88	0.36	0.04	0.60	0.07	7.15	1.83	0.84	0.11	1.28	0.21	7.08	0.22	0.45	0.20	0.25	22.59	1.82	0.00	0.00	0.12	0.03	2.16	0.45	0.00	0.00	0.20	0.02	3.58
	Scenario 2	3.13	1.24	5.87	5.48	5.14	5.85	8.19	3.20	11.18	11.34	11.88	12.78	9.49	1.35	4.64	5.37	5.37	4.27	15.60	2.33	20.03	21.23	18.77	17.18	2.26	0.58	3.57	3.37	3.44	3.58
TSMixer	Scenario 1	3.33	1.01	0.60	0.06	1.00	0.13	4.59	1.58	0.54	0.09	0.87	0.27	1.83	0.10	0.24	0.01	0.11	0.00	13.45	2.98	0.00	0.00	0.08	0.02	2.30	0.47	0.00	0.00	0.25	0.01
	Scenario 2	3.40	1.60	4.19	4.18	4.57	4.61	4.68	2.94	6.14	5.94	5.98	5.82	2.08	0.58	2.05	1.74	1.89	1.68	13.43	2.92	7.90	7.76	8.22	7.83	2.33	0.58	3.22	3.21	3.19	3.20
LightTS	Scenario 1	3.55	1.09	0.48	0.05	0.80	0.09	6.28	1.83	1.06	0.11	1.69	0.23	4.90	1.57	1.40	0.52	1.85	0.00	7.59	1.67	0.00	0.00	0.15	0.02	2.54	0.58	0.01	0.00	0.26	0.00
	Scenario 2	3.61	1.31	4.27	4.34	4.35	4.53	6.49	3.57	7.69	7.55	7.05	7.05	1.58	3.51	2.19	1.80	2.09	1.81	7.58	1.70	3.90	3.90	3.97	3.91	2.53	0.78	2.91	2.98	2.89	2.87
TimeMixer	Scenario 1	5.37	0.95	0.44	0.04	0.68	0.08	6.63	1.23	0.65	0.10	0.93	0.20	3.45	0.57	0.84	0.07	0.65	0.00	14.68	2.42	0.00	0.00	0.21	0.01	1.99	0.55	0.00	0.00	0.22	0.01
	Scenario 2	4.72	1.12	4.54	4.59	4.34	4.17	7.07	2.10	11.88	11.97	11.79	11.70	3.37	1.74	5.78	5.13	6.01	5.93	14.42	2.34	9.08	9.08	9.05	8.99	1.95	0.64	3.28	3.28	3.53	3.50
Transformer	Scenario 1	0.01	0.00	0.14	0.00	0.02	0.00	0.00	0.00	0.40	0.00	0.29	0.00	0.52	0.11	0.17	0.06	0.22	0.11	1.08	0.22	0.39	0.09	0.06	0.00	0.17	0.00	0.04	0.15	0.05	0.13
	Scenario 2	0.00	0.00	0.49	0.11	0.12	0.17	0.00	0.00	0.84	0.33	0.59	0.00	0.54	0.16	0.00	0.00	0.00	0.00	1.08	0.21	0.57	0.40	0.04	0.25	0.16	0.00	0.09	0.05	0.10	0.10
Autoformer	Scenario 1	0.27	0.18	0.05	0.05	0.03	0.08	0.02	0.05	0.06	0.08	0.09	0.28	0.56	0.30	0.14	0.14	0.20	0.11	0.34	0.00	0.08	0.04	0.00	0.04	0.04	0.05	0.07	0.00	0.00	0.07
	Scenario 2	0.22	0.18	0.17	0.22	0.19	0.22	0.10	0.17	0.34	0.25	0.42	0.45	0.59	0.19	0.53	0.41	0.53	0.41	0.44	0.00	0.57	0.32	0.27	0.34	0.06	0.08	0.08	0.04	0.02	0.00
Informer	Scenario 1	0.09	0.00	0.00	0.07	0.00	0.00	0.41	0.22	0.89	0.19	0.68	0.08	0.28	0.09	0.35	0.10	0.07	0.21	0.85	0.00	0.20	0.11	0.36	0.03	0.00	0.00	0.00	0.00	0.00	0.00
	Scenario 2	0.09	0.00	0.07	0.20	0.04	0.11	0.36	0.07	0.93	0.65	0.32	0.37	0.40	0.19	0.18	0.18	0.00	0.00	0.84	0.00	0.11	0.06	0.46	0.13	0.00	0.00	0.19	0.00	0.12	0.09
iTransformer	Scenario 1	0.52	0.08	0.04	0.00	0.05	0.02	0.28	0.05	0.01	0.00	0.04	0.00	0.85	0.05	0.13	0.02	0.03	0.02	1.33	0.09	0.01	0.00	0.00	0.00	0.07	0.00	0.00	0.00	0.00	0.00
	Scenario 2	0.51	0.11	0.33	0.35	0.31	0.28	0.28	0.09	0.59	0.61	0.49	0.48	0.84	0.32	1.35	1.08	1.18	1.08	1.33	0.09	0.57	0.56	0.60	0.64	0.07	0.00	0.11	0.12	0.10	0.11
CNN	Scenario 1	0.08	0.09	0.07	0.02	0.29	0.09	0.56	0.05	0.94	0.00	0.02	0.00	0.69	0.00	0.24	0.12	0.44	0.00	2.57	0.38	0.14	0.36	0.21	0.65	0.05	0.16	0.31	0.36	0.17	0.45
	Scenario 2	0.05	0.10	0.05	0.00	0.00	0.40	0.10	0.36	0.40	0.04	0.40	0.08	0.58	0.00	0.42	0.52	0.45	0.33	2.50	0.42	0.39	0.38	1.01	0.59	0.16	0.15	0.28	0.19	0.15	0.35
MICN	Scenario 1	0.25	0.02	0.04	0.00	0.01	0.01	0.19	0.11	0.06	0.06	0.00	0.00	1.18	0.07	0.04	0.14	0.03	1.35	0.19	0.01	0.00	0.00	0.03	0.02	0.14	0.05	0.05	0.02	0.00	0.06
	Scenario 2	0.25	0.02	0.27	0.26	0.25	0.16	0.19	0.02	0.57	0.49	0.32	0.34	1.28	0.24	1.68	1.43	1.51	1.43	1.35	0.19	0.40	0.33	0.34	0.35	0.15	0.06	0.17	0.12	0.20	0.25
SCINet	Scenario 1	0.70	0.06	0.02	0.00	0.06	0.00	0.34	0.04	0.02	0.01	0.05	0.01	0.79	0.12	0.19	0.05	0.15	0.06	1.50	0.13	0.04	0.01	0.02	0.00	0.05	0.00	0.00	0.00	0.00	0.00
	Scenario 2	0.72	0.08	0.69	0.74	0.72	0.64	0.35	0.09	0.71	0.73	0.63	0.60	0.98	0.29	1.06	1.03	1.07	0.93	1.50	0.12	1.57	1.46	1.44	1.47	0.05	0.00	0.13	0.15	0.11	0.14
TimesNet	Scenario 1	0.63	0.03	0.01	0.00	0.02	0.00	0.20	0.01	0.00	0.00	0.09	0.00	0.42	0.00	0.00	0.00	0.00	0.00	1.21	0.16	0.00	0.04	0.02	0.00	/	/	/	/	/	/
	Scenario 2	0.62	0.03	0.35	0.38	0.32	0.31	0.24	0.13	0.53	0.48	0.42	0.42	0.49	0.01	0.51	0.34	0.29	0.26	1.20	0.19	0.39	0.41	0.46	0.39	/	/	/	/	/	/

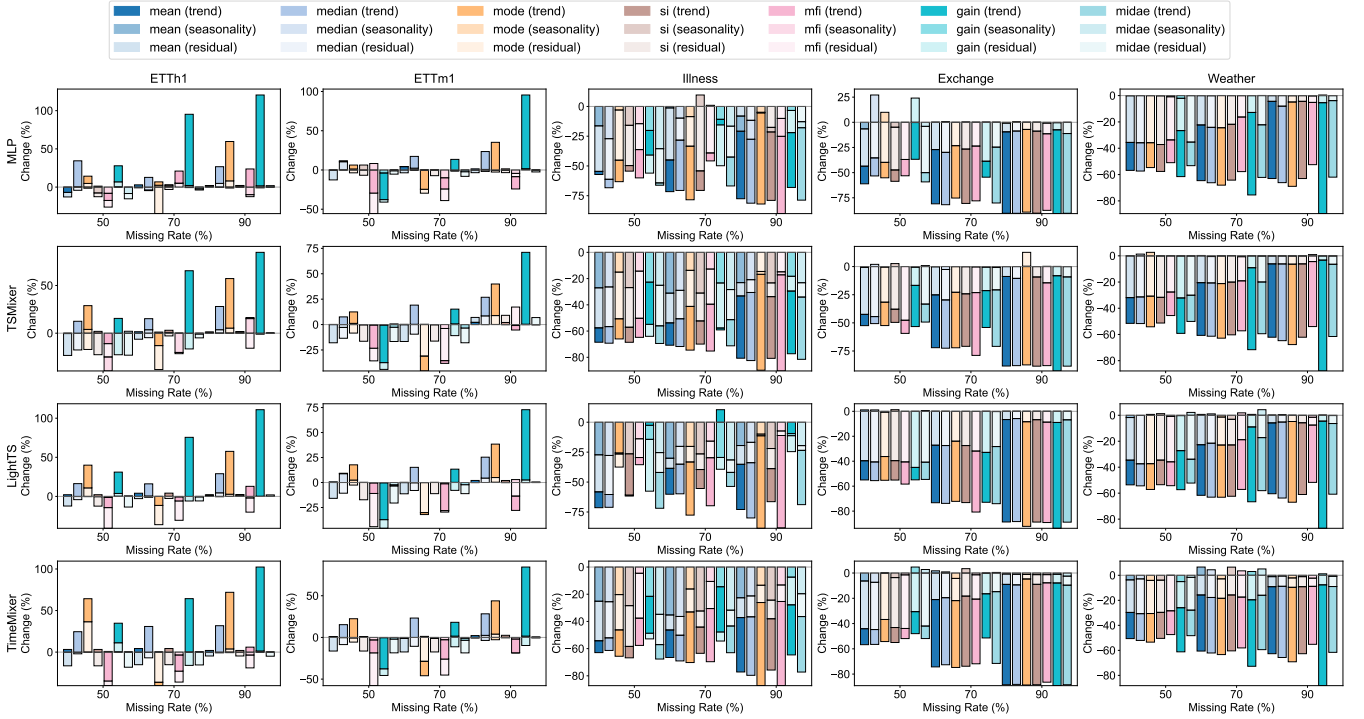


Figure 18: The percentage change in performance from pre- to post- replacement (model: MLP-based).

Summary. Finding 10. In time series imputation, methods that can effectively utilize global and local information perform better. Time series data exhibit significant local regression relationships. Therefore, imputation algorithms that can effectively model this relationship tend to achieve superior performance. ① IIM leverages both local characteristics (selecting neighborhoods) and variable dependencies (regression modeling), and it independently selects the optimal regression model for each data point. This allows it to effectively adapt to the varying characteristics of different local regions in the data, making it well-suited for handling

time series. In contrast, although MICE is also a regression-based algorithm, it builds regression models globally, resulting in slightly weaker capability in recovering local relationships. ② Tree-based methods similarly utilize global information and local structures. As noted in *Finding 5*, MissFI is less prone to overfitting compared to XGBI, leading to slightly better performance. ③ Furthermore, similarity-based methods can better preserve local consistency and thus also have advantages in processing time series.

Finding 11: Imputation quality and the recovery fidelity of components show consistent correlation. The impact strength on downstream performance is: trend > residuals > seasonality.

Table 6: Contribution rate analysis of classification tasks (model: Transformer)

Independent variables	Beers	Flights	Hospital	Red Wine	Avocado
Label correctness ratio	0.065	0.380	0.316	0.367	0.465
Class discriminability	0.797	0.701	0.738	0.485	0.635
Both of them	0.800	0.706	0.778	0.549	0.635

Table 7: Contribution rate analysis of regression tasks (model: Transformer)

Independent variables	Concrete	CCPP	Airfoil	Abalone	ParisHP
Imputation bias	0.536	0.970	0.874	0.477	0.625
Feature-target correlation	0.357	0.207	0.253	0.201	0.516
Both of them	0.546	0.976	0.874	0.484	0.632

Table 8: Contribution rate analysis of classification tasks (model: CNN)

Independent variables	Beers	Flights	Hospital	Red Wine	Avocado
Label correctness ratio	0.054	0.381	0.396	0.362	0.440
Class discriminability	0.833	0.736	0.811	0.485	0.590
Both of them	0.834	0.745	0.832	0.546	0.590

Table 9: Contribution rate analysis of regression tasks (model: CNN)

Independent variables	Concrete	CCPP	Airfoil	Abalone	ParisHP
Imputation bias	0.692	0.974	0.793	0.654	0.643
Feature-target correlation	0.510	0.211	0.243	0.370	0.535
Both of them	0.694	0.980	0.793	0.655	0.649

Better imputation quality corresponds to stronger correlations across all components. Imputation methods that has better imputation quality also align with those showing higher correlation coefficients. Although decomposition is known to aid forecasting by revealing latent patterns [13, 92], prior work does not rank component importance. In contrast, our analysis reveals that, under equal perturbation, trend has the greatest impact, which aligns with the fact that many deep time series models emphasize capturing long-term patterns. Similarly, our finding of residuals also supports the perspective in [3] that residuals require careful consideration. The long-term, gradual trend is decisive since it determines the overall direction of the series. Next, residuals rank second because its unstructured and irregular nature is difficult for MLP to model. Finally, seasonality, with its clearly structured and repetitive features, is easier for MLP to capture, resulting in the smallest impact.

Finding 12. Time series exhibits component interaction effect: any component change triggers global reconstruction, especially when all components are dirty. Therefore, optimizing only specific components leads to uncertain task performance. The *component interaction effect* describes the coupling among trend, seasonality, and residuals, where a change to one propagates to others due to decomposition algorithms or data dependencies. *Global reconstruction* refers to the resulting update of the whole series, as corroborated in [48]. Our analysis of three D_{pro} datasets confirm the findings. The results of scenario 2 reveal that the component interaction effect significantly undermines the effectiveness of single optimization. Due to inter-component coupling, optimizing a single component may trigger degradation in others, thus not guaranteeing global improvement and making performance uncertain. Because of this, replacing components like seasonality and residuals fails to yield better results since their values are volatile. While the trend changes more steadily, replacing it helps to achieve a relatively stable state.

5.6 Extended Experiments

To assess the reliability and generalizability of MLP-based findings, we validate them on two additional basic models: Transformer and CNN. Furthermore, we extend experiments to 9 deep time series models, including MLP-based models (TSMixer [11], LightTS [107], TimeMixer [91]), Transformer-based models (Autoformer [96], Informer [111], iTransformer [58]), and CNN-based models (MICN [89], SCINet [57], TimesNet [95]).

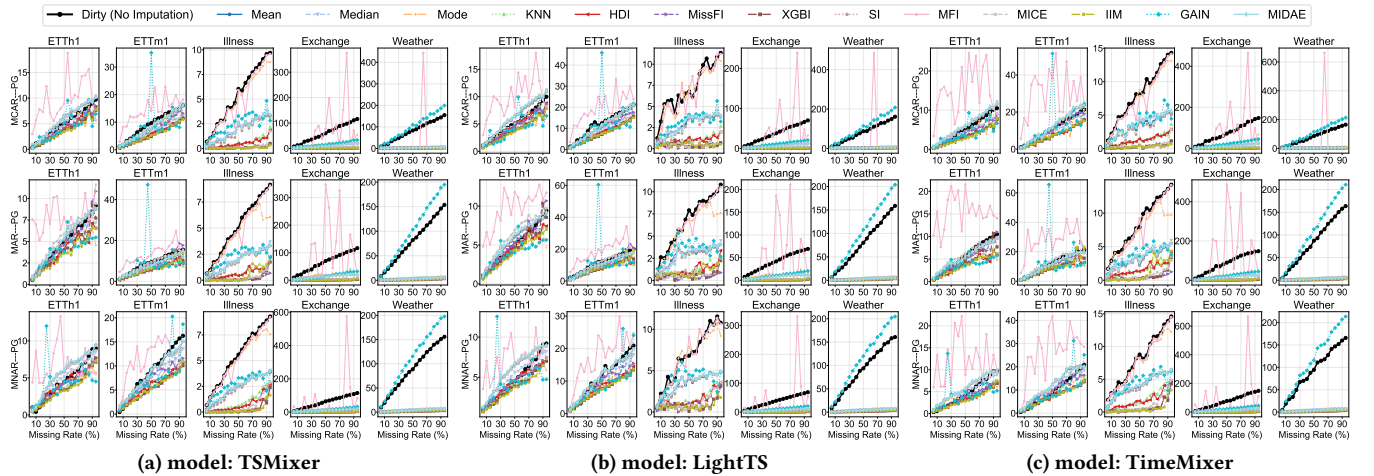
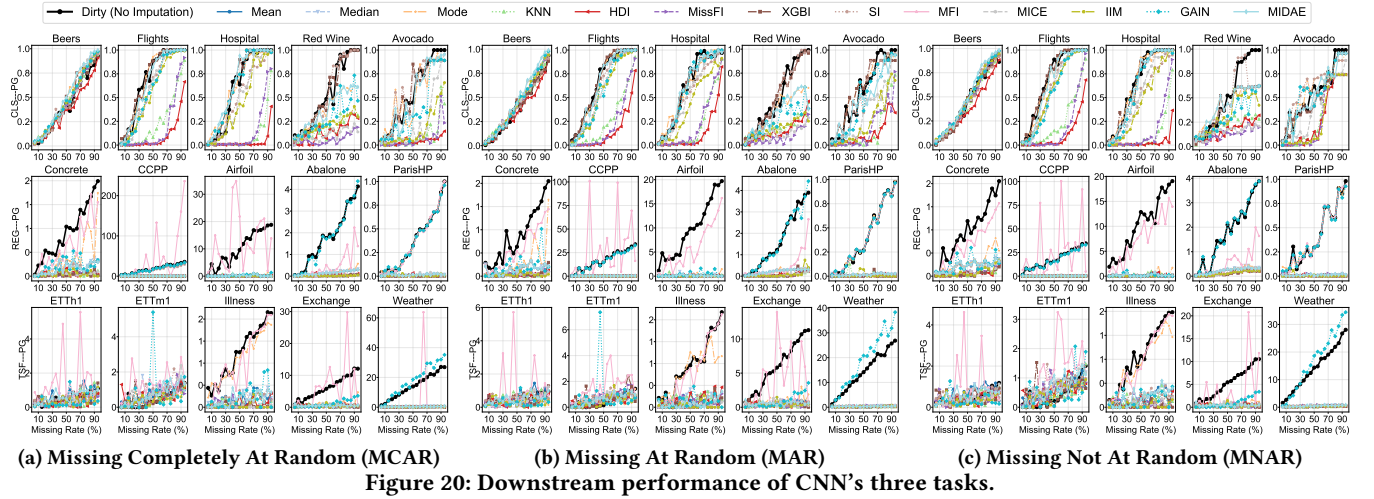
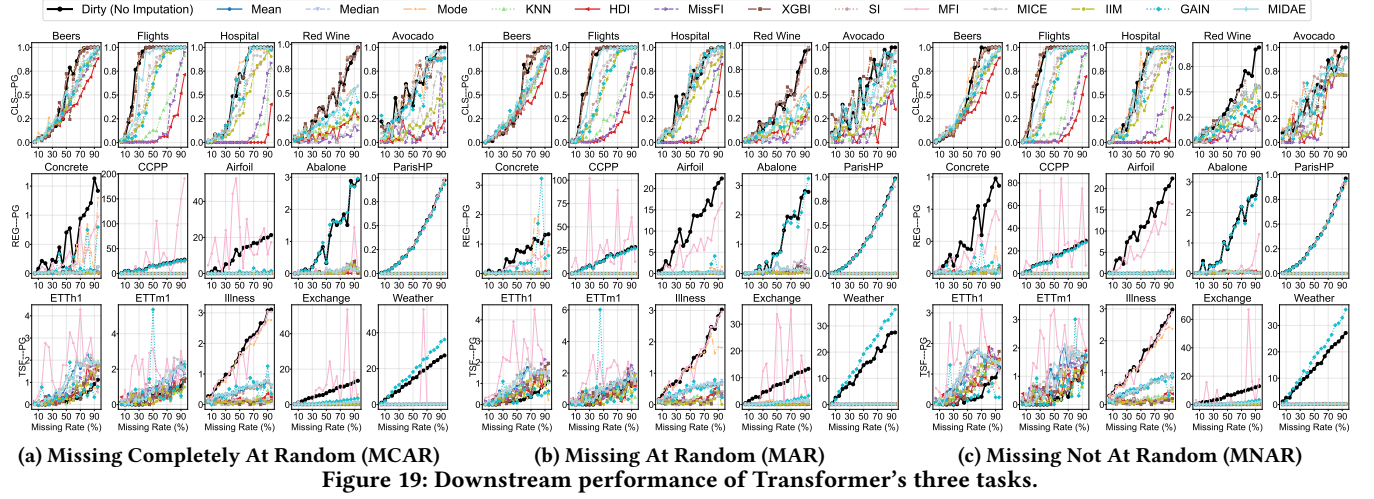
TSMixer is a fully MLP-based architecture that achieves time series forecasting by stacking MLP layers. It includes two types of operations: time mixing and feature mixing. The time mixing operation applies fully connected layers in the time domain and is shared across all features, while the feature mixing operation captures covariate information and is shared across time steps. Residual connections are applied between time mixing and feature mixing. Additionally, the model includes components such as temporal projection and normalization. By fully capturing temporal and covariate information, the model accomplishes time series forecasting.

LightTS is a lightweight deep learning architecture designed for multivariate time series forecasting by capturing complex temporal patterns and dependencies among different variables. For temporal pattern extraction, it designs two down-sampling strategies: continuous sampling and interval sampling. Continuous sampling captures short-term local patterns, while interval sampling captures long-term global patterns. Building on this, LightTS designs an information exchange block based on MLP with a bottleneck architecture to facilitate information exchange across downsampled subsequences and time steps, thereby achieving time series forecasting.

TimeMixer is a deep learning architecture based on multi-scale mixing, featuring two modules: Past-Decomposable-Mixing (PDM) and Future-Multipredictor-Mixing (FMM). PDM aggregates microscopic seasonal information and macroscopic trend information, while FMM leverages the complementary forecasting capabilities of multi-scale observations. This enables efficient long-term or short-term time series forecasting.

Autoformer is a deep decomposition architecture for long-term forecasting. It integrates time series decomposition as a built-in module into the model, progressively extracting and optimizing predictions through a gradual decomposition mechanism. Leveraging an auto-correlation mechanism, it discovers and aggregates period-based dependencies at the subseries level via the FFT, thereby improving computational efficiency and achieving superior information utilization compared to traditional self-attention mechanisms.

Informer is an efficient model designed for long-term forecasting. Building on the standard Transformer, it introduces the ProbSparse self-attention mechanism, which identifies key dot-product pairs based on query sparsity to reduce complexity. It also incorporates a self-attention distillation operation to progressively shorten the sequence length and a generative decoder to produce long-horizon predictions in a single step, enhancing both computational efficiency and forecasting accuracy in long-range forecasting scenarios.



iTransformer independently embeds the time series of each variable into a "variate token". It employs a self-attention mechanism to capture inter-variable correlations and uses a feed-forward network to learn nonlinear representations of each variable's sequence. Without modifying any native Transformer components,

it effectively models both multivariate correlations and sequential dependencies by simply reorienting the operational dimensions.

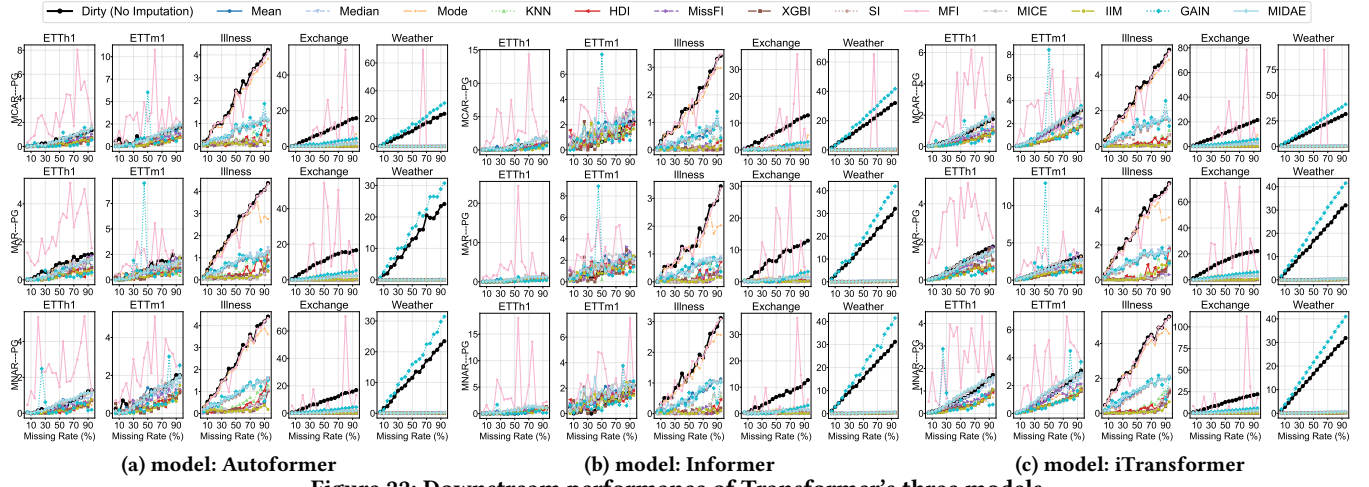


Figure 22: Downstream performance of Transformer's three models.

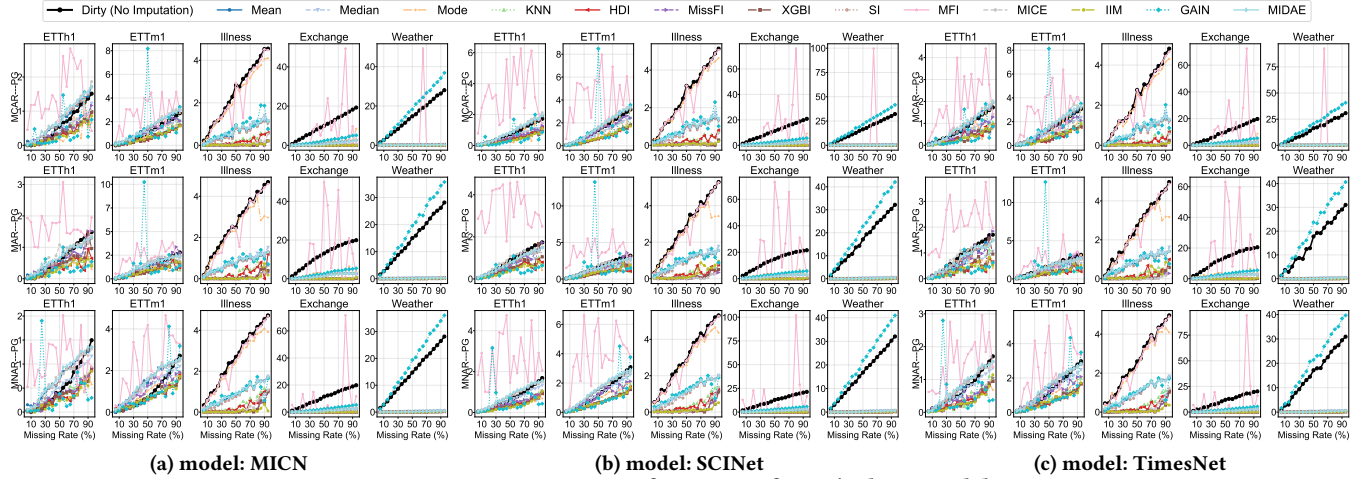


Figure 23: Downstream performance of CNN's three models.

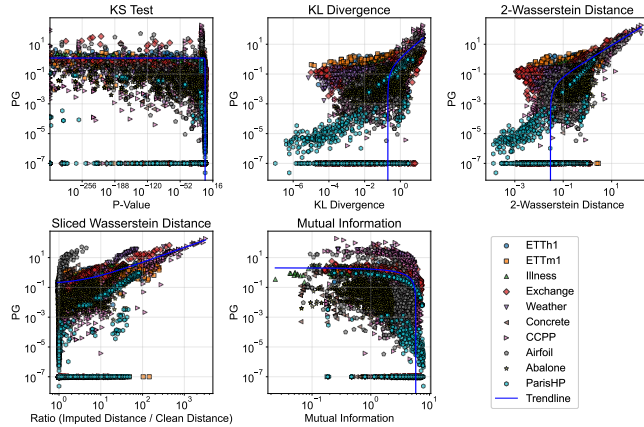


Figure 24: The distribution of quality and performance (model: Transformer).

MICN models latent patterns at different scales in a time series through a multi-branch architecture. Each branch uses downsampling convolutions to extract local features and then applies equidistant convolutions to capture global correlations, thereby effectively fusing both local and global information.

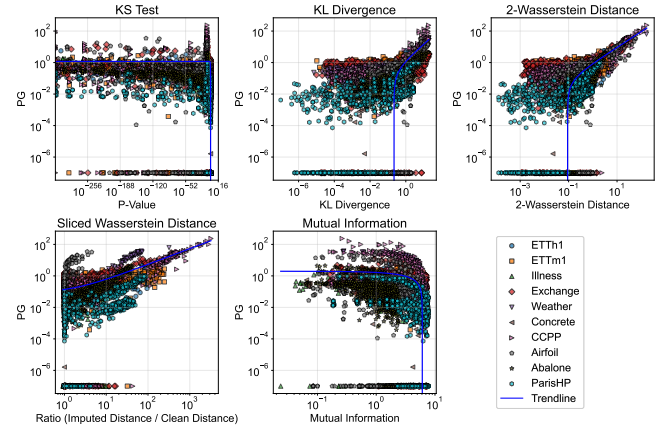


Figure 25: The distribution of quality and performance (model: CNN).

TimesNet is a general-purpose model for time series analysis. It adaptively discovers multi-periodicity in time series via the FFT, and reshapes the one-dimensional time series into multiple two-dimensional tensors based on different period lengths. Within each

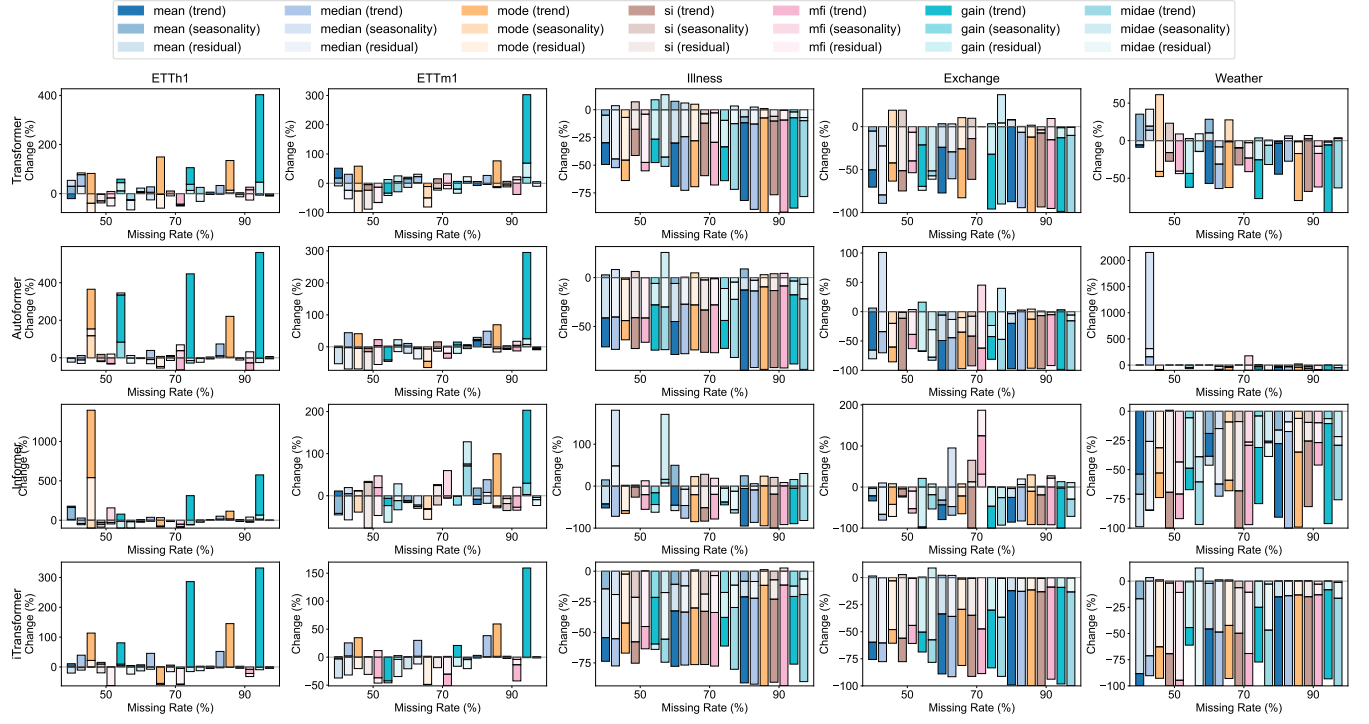


Figure 26: The percentage change in performance from pre- to post-replacement (model: Transformer-based).

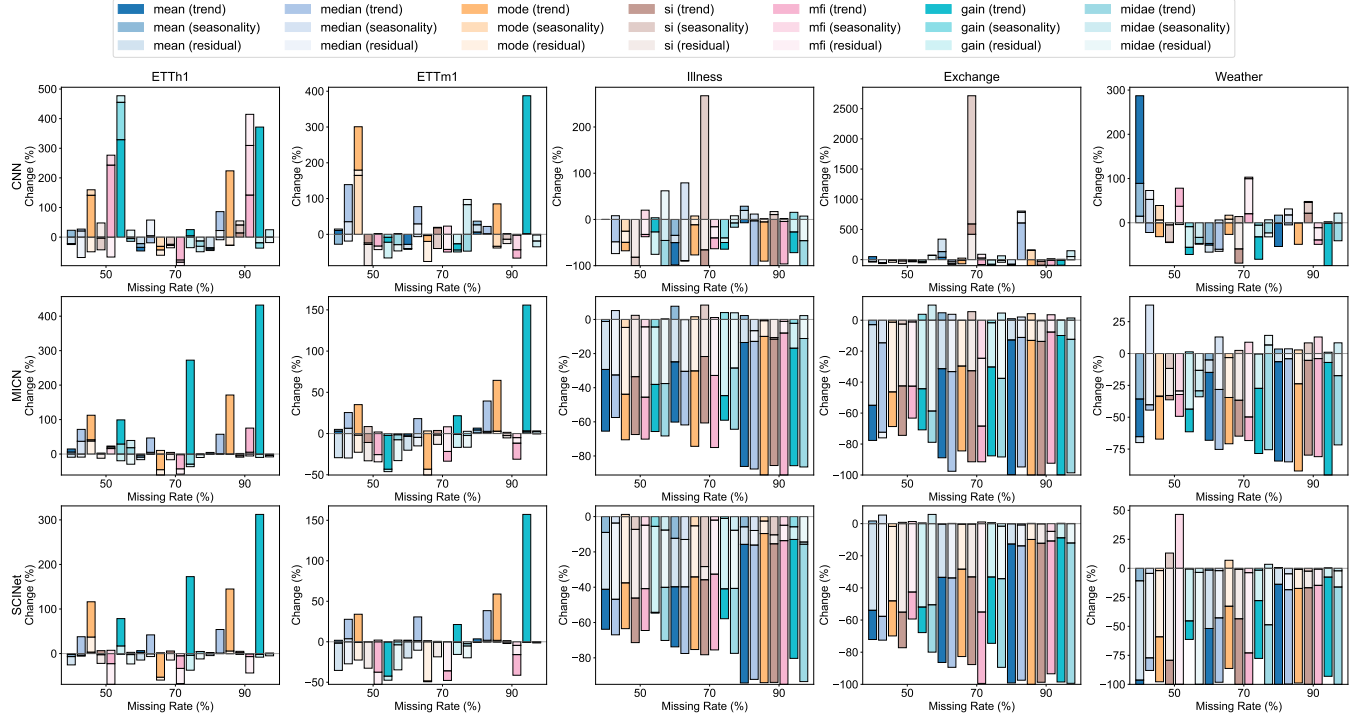


Figure 27: The percentage change in performance from pre- to post-replacement (model: CNN-based).

2D structure, intra-period variations correspond to columns and inter-period trends correspond to rows, enabling a 2D convolutional network to model both types of changes simultaneously.

SCINet is a time series forecasting model based on sample convolution and interaction. Its core design recursively downsamples the input sequence into two subsequences according to odd/even

positions, uses separate convolutional kernels to extract features from each, and then employs an interactive learning mechanism to enable information complementation between them. Through a hierarchical "downsampling-convolution-interaction" architecture, it aggregates rich temporal features from multiple time resolutions, thereby capturing complex short-term and long-term dependencies.

Results. The performance results are shown in Figure 19, Figure 20, Figure 21, Figure 22 and Figure 23. The results of imputation quality and downstream performance are presented in Figure 24 and Figure 25. Table 5 presents the performance for different c_T under scenario 1 and scenario 2, respectively. The results of contribution rate analysis are presented in Table 6, Table 7, Table 8 and Table 9. The results of percentage change in performance from pre- to post- replacement are illustrated in Figure 18, Figure 26 and Figure 27.

Summary. Finding 13. Insights from basic models are transferable to complex, architecturally homologous deep models. Taking MLP as an example, despite structural differences, all these deep models rely fundamentally on MLP networks to capture trend, seasonality, and residuals. This characteristic remains unchanged with increased model complexity. Despite specific mechanisms to enhance the ability to learn complex time series patterns, their central goal remains the same: to capture autoregressive time-step relations and regression dependencies among covariates, with a shared emphasis on modeling trend and seasonality. This shared foundation and objective stabilizes the impact of imputation and allows findings to transfer across models. Notably, to our knowledge, existing research on transferring findings across models is only studies by OpenAI [44] and DeepMind [39]. However, our study represents the first effort to transfer findings from a foundational model to complex architecturally homologous ones. Further exploration is still needed to establish criteria for such transferability.

6 RECOMMENDATIONS

6.1 Basic recommendations

Based on the IIKP-Path and the findings obtained, we provide the recommendations for imputation method selection. As shown in Figure 28, they include four aspects.

(1) **Data-algorithm pre-screening.** ① **Missing Rate.** At low missing rates, all algorithms are viable. Tree-based methods are preferred due to its ability to handle complex data. Statistical methods perform relatively poorly and are not recommended. At high missing rates, tree- and regression-based methods are recommended since they perform reliably. DL-based methods require substantial data and statistical methods ignore variable relationships, all unsuitable here. ② **Data Patterns.** Strong variable correlations. Regression-based methods are recommended to leverage these dependencies. Statistical methods may lose relevant information. Complex non-linear relationships. DL-based methods capture nonlinear patterns well. Compression-based methods cannot model them. Significant local similarity. Similarity-based methods exploit local proximity. Statistical and compression-based methods ignore local relationships and are unsuitable. ③ **Time Cost:** Prioritize algorithms with time costs at Level 1 or 2. MissFI is acceptable at low missing rates. Avoid algorithms at Level 3 or 4. Statistical methods and SI have low time costs but show poor performance and are not recommended.

(2) **Quality-assisted filtering.** ① Although no imputation algorithm is universally best, similarity-, tree-, and regression-based methods generally achieve better results. Statistical, compression- and DL-based methods often underperform. ② Although imputation quality and downstream performance are not strictly one-to-one, their fundamental connection cannot be ignored. Improving data quality remains crucial, and selecting suitable imputation algorithms is an effective step toward that goal.

(3) **Task-key factor alignment.** ① *classification*, prioritize KNN, HDI, and MissFI. ② *regression*, prioritize tree- and regression-based methods, with similarity-based methods be supplementary. ③ *time series forecasting*, similarity-, tree-, and regression-based methods are recommended. IIM and MissFI are worth prioritizing.

(4) **Optimization and risk warnings.** ① [2] notes a trend toward hybrid imputation systems, making algorithm combinations a viable direction. Boundary samples can be filtered for manual verification post-imputation since classification is sensitive to them. ② Although class discriminability dominates classification performance, forcibly enhancing it through label manipulation may harm performance. Similarly, optimizing individual time series components has uncertain downstream effects, warranting caution.

6.2 Pruning recommendation method

The workflow of the pruning recommendation method is illustrated in Figure 29. Before application, construct the knowledge base following the aforementioned steps for building the IIKP-Path. During the application phase, the inputs are missing dataset D_{miss} , performance threshold σ and downstream task T . The output is the set of viable imputation algorithms $\mathcal{P}_3$, with two intermediate results generated during the process: the filtered imputation algorithm sets after each pruning step, denoted as $\mathcal{P}_1$ and $\mathcal{P}_2$.

Given that the effectiveness of each imputation algorithm on D_{miss} is initially unknown, relying solely on downstream task performance screening makes it difficult to guarantee with high probability that the "selected imputation algorithm is a viable solution (i.e., meets the threshold)". Therefore, in addition to task performance, we incorporate the data quality and key factors from IIKP-Path. i.e., we perform three rounds of pruning based on downstream task performance, data quality metrics, and key factors, respectively. Later, we will see that $\mathcal{P}_3$ significantly reduces the probability of non-viable solutions compared to $\mathcal{P}_1$, demonstrating the effectiveness of introducing these rounds. Below, we describe the threshold settings and pruning operations, then we provide experimental validation.

Threshold setting. For a given performance threshold σ , set the performance thresholds σ_1 , σ_2 , and σ_3 for three pruning steps based on each one's requirement. Using the data from the knowledge base, establish a mapping between the performance thresholds σ_1 , σ_2 , σ_3 and the evaluation thresholds τ_1 , τ_2 , τ_3 . Here, $\tau_1 = \sigma_1$; τ_2 is defined as the upper or lower bound of the data quality metrics of imputation algorithms whose performance can meet σ_2 ; τ_3 is defined as the upper or lower bound of the key factor metrics of imputation algorithms whose performance can meet σ_3 . Consequently, when the knowledge base is updated, the mapping relationships are also updated synchronously, thereby achieving synchronous updates of the evaluation thresholds. Furthermore, to minimize the

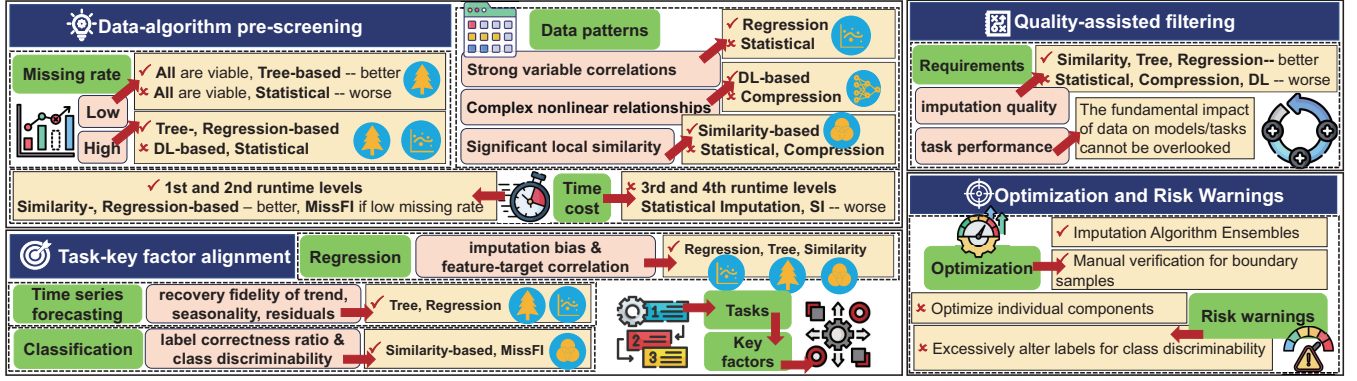


Figure 28: The workflow of basic recommendations.

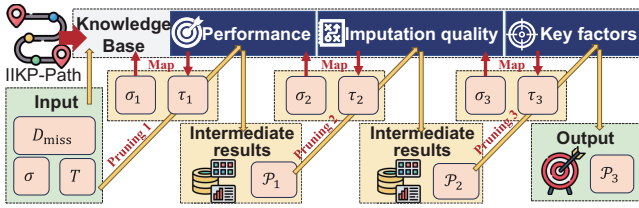


Figure 29: The pruning recommendation method workflow.

probability of including non-viable solutions, in addition to setting $\sigma_1 = \sigma$, we also require $\sigma_2 \leq \sigma_1$ and $\sigma_3 \leq \sigma_2$.

Pruning. Various industries and fields requiring data analysis possess substantial amounts of historical domain data, which can be fully utilized during the pruning. Before pruning begins, the historical data for D_{miss} is analyzed according to the steps for constructing the IIKP-Path, obtaining results for each step. Subsequently, perform the following operations. Based on the performance evaluation results, imputation algorithms that cannot meet the threshold τ_1 are identified and removed, resulting in $\mathcal{P}_1$. Taking the intermediate result $\mathcal{P}_1$ as input, imputation algorithms within it that cannot meet the threshold τ_2 are identified and removed, resulting in $\mathcal{P}_2$. Finally, taking the intermediate result $\mathcal{P}_2$ as input, imputation algorithms within it that cannot meet the threshold τ_3 are identified and removed, resulting in $\mathcal{P}_3$, which serves as the Output.

Experimental validation. We utilize all experimental results obtained in Section 5 as the knowledge base. Then, we validate the pruning-based imputation algorithm selection method using datasets Glass for classification, BostonHP for regression, and ETTh2 for time series forecasting. For each dataset, 30% of the data is treated as historical data, and the remaining data serves as test data. The partitioning method is as follows: for classification datasets, 30% of each class is randomly selected; for regression datasets, 30% is randomly selected; for time series datasets, the first 30% is selected. The missing value injection step is consistent with the previous description. Grid parameters are set for $\sigma_1, \sigma_2, \sigma_3$: σ_1 ranges from 0.05 to 0.95 with step size 0.05; σ_2 ranges from $\max(0.01, (\sigma_1 - 0.05))$ to σ_1 with step size 0.01; σ_3 ranges from $\max(0.01, (\sigma_2 - 0.05))$ to σ_2 with step size 0.01. The output of each dataset under different parameter settings is obtained. Simultaneously, downstream task performance evaluation experiments are conducted to obtain the

real sets of viable imputation algorithms, which are then compared with the output of the pruning-based method.

For each dataset, there are 19 possible values for the missing rate α and 19 for σ_1 , resulting in a grid defined by α and σ_1 with $19 \times 19 = 361$ cells. However, for Glass, the case of $\alpha = 95\%$ is not considered because MICE fails to converge, resulting in $18 \times 19 = 342$ grid cells. For each grid cell (i.e., under conditions where both α and σ_1 are fixed), there are at most $6 \times 6 = 36$ possible combinations of σ_2 and σ_3 . The number of errors in the output under the best threshold setting among these 36 possibilities is compared with the real set of viable imputation algorithms. Finally, results across all grid cells are shown in Table 10.

Accuracy. The experimental results demonstrate that, under reasonable threshold settings, the proportion of cases with an absolute error count of 3 or fewer between the pruning-based method's output and the true set of feasible imputation algorithms is 68.71%, 78.12%, and 88.37%, respectively. Furthermore, when compared with results relying solely on historical performance data, we find that introducing the knowledge base, data quality round, and key factors round significantly reduces the proportion of outputs containing non-viable solutions from 37.72%, 37.40%, and 26.32% to 28.36%, 31.58%, and 17.73%, respectively.

Efficiency. Since the construction of the knowledge base and the analysis of historical data for D_{miss} can be completed before application, leveraging experience from the knowledge base enhances efficiency during application compared to exhaustive traversal. To obtain the viable imputation algorithm set and apply it to the target task, two stages must be completed: data imputation and task application. Assuming there are n imputation algorithms, the average time complexity for each imputation algorithm is $O(t_{p\alpha})$, and the average time complexity for the downstream task is $O(t_{T\alpha})$. In the case of exhaustive traversal of all imputation algorithms, the data imputation stage requires $O(n \cdot t_{p\alpha})$. Since the performance of each imputation algorithm on the task is unknown, the task application stage also requires $O(n \cdot t_{T\alpha})$, resulting in an overall average time complexity of $O(n \cdot (t_{p\alpha} + t_{T\alpha}))$. However, after applying the pruning-based method, m (where $m \leq n$) imputation algorithms can be selected based on the knowledge base. If all of these are subjected to performance testing, the required time is only $O(m \cdot (t_{p\alpha} + t_{T\alpha}))$. If only the best algorithm is selected for testing ($m = 1$), the time cost becomes even smaller, merely $O(t_{p\alpha} + t_{T\alpha})$.

Table 10: The output error count of pruning method

Error count	0-1	2-3	≥ 4
Glass	127(37.13%)	108(31.58%)	107(31.29%)
BostonHP	202(55.96%)	80(22.16%)	79(21.88%)
ETTh2	299(82.83%)	20(5.54%)	42(11.63%)

On the three aforementioned datasets, the average values of m are 5.96, 9.43, and 0.96, corresponding to time cost reductions of 40.40%, 27.46%, and 92.62%, respectively, compared to exhaustive traversal.

7 CONCLUSION

In this paper, we propose **IIKP-Path**, a four-stage empirical framework, spanning imputation algorithm analysis, imputation quality assessment, key factor analysis, and performance evaluation, to systematically study how missing data imputation affects MLP performance across diverse analytics tasks. Grounded in the interplay of imputation, data quality, and model/task perspectives, we summarize 13 key findings that reveal the intricate coupling among these elements and inform practical recommendations. Looking ahead, we plan to extend this investigation beyond MLPs to more complex and realistic models, to assess the generalizability of our insights.

REFERENCES

- [1] Mohamed Abdelal, Christian Hammacher, and Harald Schöning. 2023. REIN: A Comprehensive Benchmark Framework for Data Cleaning Methods in ML Pipelines. In *Proceedings 26th International Conference on Extending Database Technology, EDBT 2023, Ioannina, Greece, March 28-31, 2023*, Julia Stoyanovich, Jens Teubner, Nikos Mamoulis, Evaggelia Pitoura, Jan Mühlig, Katja Hose, Sourav S. Bhowmick, and Matteo Lissandrini (Eds.). OpenProceedings.org, 499–511. doi:10.48786/EDBT.2023.43
- [2] Deepak Adhikari, Wei Jiang, Jinyu Zhan, Zhiyuan He, Danda B. Rawat, Uwe Aickelin, and Hadi Akbarzade Khorshidi. 2023. A Comprehensive Survey on Imputation of Missing Data in Internet of Things. *ACM Comput. Surv.* 55, 7 (2023), 133:1–133:38. doi:10.1145/3533381
- [3] Reza Asadi and Amelia C. Regan. 2020. A spatio-temporal decomposition based deep neural network for time series forecasting. *Appl. Soft Comput.* 87 (2020), 105963. doi:10.1016/j.asoc.2019.105963
- [4] Karima Benhamza, Mohamed Salah Benselim, Hanane Naidja, and Hamid Seridi. 2025. A comprehensive survey of imputation methods in medical missing data analysis. *Appl. Intell.* 55, 11 (2025), 778. doi:10.1007/S10489-025-06602-2
- [5] Konstantinos Benidis, Syama Sundar Rangapuram, Valentin Flunkert, Yuyang Wang, Danielle C. Maddix, Ali Caner Türkmen, Jan Gasthaus, Michael Bohlke-Schneider, David Salinas, Lorenzo Stella, François-Xavier Aubet, Laurent Colloot, and Tim Januschowski. 2023. Deep Learning for Time Series Forecasting: Tutorial and Literature Survey. *ACM Comput. Surv.* 55, 6 (2023), 121:1–121:36. doi:10.1145/3533382
- [6] James Bergstra and Yoshua Bengio. 2012. Random Search for Hyper-Parameter Optimization. *J. Mach. Learn. Res.* 13 (2012), 281–305. doi:10.5555/2503308.2188395
- [7] George EP Box, Gwilym M Jenkins, Gregory C Reinsel, and Greta M Ljung. 2015. *Time series analysis: forecasting and control*. John Wiley & Sons.
- [8] Carla E. Brodley and Mark A. Friedl. 1999. Identifying Mislabeled Training Data. *J. Artif. Intell. Res.* 11 (1999), 131–167. doi:10.1613/JAIR.606
- [9] Pope D. Brooks, Thomas and Michael Marcolini. 1989. Airfoil Self-Noise. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5VW2C>.
- [10] Jialei Chen, Yuanbo Xu, Pengyang Wang, and Yongjian Yang. 2023. Deep Generative Imputation Model for Missing Not At Random Data. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, CIKM 2023, Birmingham, United Kingdom, October 21-25, 2023*, Ingo Frommholz, Frank Hopfgartner, Mark Lee, Michael Oakes, Mounia Lalmas, Min Zhang, and Rodolfo L. T. Santos (Eds.). ACM, 316–325. doi:10.1145/3583780.3614835
- [11] Si-An Chen, Chun-Liang Li, Nate Yoder, Sercan Ö. Arik, and Tomas Pfister. 2023. TSMixer: An all-MLP Architecture for Time Series Forecasting. *CoRR abs/2303.06053* (2023). doi:10.48550/ARXIV.2303.06053 arXiv:2303.06053
- [12] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016*, Balaji Krishnapuram, Mohak Shah, Alexander J. Smola, Charu C. Aggarwal, Dou Shen, and Rajeev Rastogi (Eds.). ACM, 785–794. doi:10.1145/2939672.2939785
- [13] Mingyue Cheng, Zhiding Liu, Xiaoyu Tao, Qi Liu, Jintao Zhang, Tingyue Pan, Shilong Zhang, Panjing He, Xiaohan Zhang, Daoyu Wang, et al. 2025. A comprehensive survey of time series forecasting: Concepts, challenges, and future directions. *Authorea Preprints* (2025).
- [14] Robert B Cleveland, William S Cleveland, Jean E McRae, Irma Terpenning, et al. 1990. STL: A seasonal-trend decomposition. *J. off. Stat* 6, 1 (1990), 3–73.
- [15] Jacob Cohen. 1968. Multiple regression as a general data-analytic system. *Psychological bulletin* 70, 6p1 (1968), 426.
- [16] Jacob Cohen, Patricia Cohen, Stephen G West, and Leona S Aiken. 2013. *Applied multiple regression/correlation analysis for the behavioral sciences*. Routledge.
- [17] Corinna Cortes and Vladimir Vapnik. 1995. Support-Vector Networks. *Mach. Learn.* 20, 3 (1995), 273–297. doi:10.1007/BF00994018
- [18] Cerdeira A. Almeida F. Matos T. Cortez, Paulo and J. Reis. 2009. Wine Quality. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C56S3T>.
- [19] Estela Bee Dagum and Silvia Bianconcini. 2016. *Seasonal adjustment methods and real time trend-cycle estimation*. Vol. 8. Springer.
- [20] Abhimanyu Das, Weihao Kong, Andrew Leach, Shaan Mathur, Rajat Sen, and Rose Yu. 2023. Long-term Forecasting with TiDE: Time-series Dense Encoder. *Trans. Mach. Learn. Res.* 2023 (2023). <https://openreview.net/forum?id=pCbC3aQB5W>
- [21] Amal Dev. 2025. Avocado Ripeness Classification Dataset. <https://www.kaggle.com/datasets/amldvvs/avocado-ripeness-classification-dataset/data> Kaggle dataset.
- [22] Xiaohan Ding, Xiangyu Zhang, Jungong Han, and Guiguang Ding. 2021. RepMLP: Re-parameterizing Convolutions into Fully-connected Layers for Image Recognition. *CoRR abs/2105.01883* (2021). arXiv:2105.01883 <https://arxiv.org/abs/2105.01883>
- [23] Dalibor Dobrilovic, Razvan Bogdan, Visnja Ognjenovic, and Marius Marcu. 2023. Analyses on Usage of MLP Regression with WSN Data for Predicting Room Occupancy. In *19th IEEE International Conference on Intelligent Computer Communication and Processing, ICCP 2023, Cluj-Napoca, Romania, October 26-28, 2023*, Sergiu Nedeveschi, Rodica Potolea, and Radu Razvan Slavescu (Eds.). IEEE, 131–136. doi:10.1109/ICCP60212.2023.10398671
- [24] Elvis Dohmatob, Mohammad Pezesghi, and Reyhane Askari Hemmat. 2025. Why Less is More (Sometimes): A Theory of Data Curation. *CoRR abs/2511.03492* (2025). doi:10.48550/ARXIV.2511.03492 arXiv:2511.03492
- [25] Bradley Efron, Trevor Hastie, Iain Johnstone, and Robert Tibshirani. 2004. Least angle regression. (2004).
- [26] Isil Baysal Erez, Jan Flokstra, Mannes Poel, and Maurice van Keulen. 2024. The Impact of Missing Data Imputation on Model Performance and Explainability. In *BNAIC/BeNeLearn 2024: Joint International Scientific Conference on AI and Machine Learning*.
- [27] Ronald A Fisher. 1936. The use of multiple measurements in taxonomic problems. *Annals of eugenics* 7, 2 (1936), 179–188.
- [28] Keinosuke Fukunaga. 2013. *Introduction to statistical pattern recognition*. Elsevier.
- [29] Pedro J. García-Laencina, José-Luis Sancho-Gómez, and Aníbal R. Figueiras-Vidal. 2010. Pattern classification with missing data: a review. *Neural Comput. Appl.* 19, 2 (2010), 263–282. doi:10.1007/S00521-009-0295-6
- [30] B. German. 1987. Glass Identification. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5WW2P>.
- [31] Lovedeep Gondara and Ke Wang. 2018. MIDA: Multiple Imputation Using Denoising Autoencoders. In *Advances in Knowledge Discovery and Data Mining - 22nd Pacific-Asia Conference, PAKDD 2018, Melbourne, VIC, Australia, June 3-6, 2018, Proceedings, Part III (Lecture Notes in Computer Science, Vol. 10939)*, Dinh Q. Phung, Vincent S. Tseng, Geoffrey I. Webb, Bao Ho, Mohadesesh Ganji, and Lida Rashidi (Eds.). Springer, 260–272. doi:10.1007/978-3-319-93040-4_21
- [32] Youdi Gong, Guangzhen Liu, Yunzhi Xue, Rui Li, and Lingzhong Meng. 2023. A survey on dataset quality in machine learning. *Inf. Softw. Technol.* 162 (2023), 107268. doi:10.1016/j.infsof.2023.107268
- [33] Albert Gu and Tri Dao. 2023. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *CoRR abs/2312.00752* (2023). doi:10.48550/ARXIV.2312.00752 arXiv:2312.00752
- [34] Feng Guo, Jianwei Wang, Ying Zhang, Kai Wang, and Xuemin Lin. 2024. Missing Data Imputation with Uncertainty-Driven Network. *Proceedings of the ACM on Management of Data* 2, 3, Article 198 (2024). doi:10.1145/3654920
- [35] Meng-Hao Guo, Zheng-Ning Liu, Tai-Jiang Mu, and Shi-Min Hu. 2023. Beyond Self-Attention: External Attention Using Two Linear Layers for Visual Tasks. *IEEE Trans. Pattern Anal. Mach. Intell.* 45, 5 (2023), 5436–5447. doi:10.1109/TPAMI.2022.3211006
- [36] Denys Herasymuk, Falaah Arif Khan, and Julia Stoyanovich. 2024. Responsible Model Selection with Virny and VirnyView. In *Companion of the 2024 International Conference on Management of Data, SIGMOD/PODS 2024, Santiago, Chile, June 9-15, 2024*, Pablo Barceló, Nayat Sánchez-Pi, Alexandra Meliou, and S. Sudarshan (Eds.). ACM, 488–491. doi:10.1145/3626246.3654738
- [37] Ryan Hildebrandt, Rahul Atul Bhoje, Sharad Mehrotra, Christopher Tull, and Nalini Venkatasubramanian. 2025. DIM-SUM: Dynamic Imputation for Smart

- Utility Management. *Proc. VLDB Endow.* 18, 11 (2025), 4451–4464. doi:10.14778/3749646.3749705
- [38] Vitali Hirsch, Peter Reimann, Dennis Treder-Tschechlov, Holger Schwarz, and Bernhard Mitschang. 2023. Exploiting domain knowledge to address class imbalance and a heterogeneous feature space in multi-class classification. *VLDB J.* 32, 5 (2023), 1037–1064. doi:10.1007/S00778-023-00780-6
- [39] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack W. Rae, and Laurent Sifre. 2022. An empirical analysis of compute-optimal large language model training. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/c1e2faff6f588870935f114be04a3e5-Abstract-Conference.html
- [40] Yang Hu, Haoxuan You, Zhecan Wang, Zhicheng Wang, Erjin Zhou, and Yue Gao. 2021. Graph-MLP: Node Classification without Message Passing in Graph. *CoRR abs/2106.04051* (2021). arXiv:2106.04051 <https://arxiv.org/abs/2106.04051>
- [41] José M. Jerez, Ignacio Molina, Pedro J. Garcia-Laencina, Emilio Alba, Nuria Ribelles, Miguel Martín, and Leonardo Franco. 2010. Missing data imputation using statistical and machine learning methods in a real breast cancer problem. *Artif. Intell. Medicine* 50, 2 (2010), 105–115. doi:10.1016/J.ARTMED.2010.05.002
- [42] Luke Oluwaseye Joel, Wesley Doorsamy, and Babu Sena Paul. 2022. A review of missing data handling techniques for machine learning. *International Journal of Innovative Technology and Interdisciplinary Sciences* 5, 3 (2022), 971–1005.
- [43] Ranjeet B. Kagade and N. Vijayaraj. 2025. OCCOA for clustering-based intrusion detection system with MLP-RNN architecture. *Multim. Tools Appl.* 84, 11 (2025), 8567–8592. doi:10.1007/S11042-024-19107-W
- [44] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. *CoRR abs/2001.08361* (2020). arXiv:2001.08361 <https://arxiv.org/abs/2001.08361>
- [45] Bojan Karlas, Babak Salimi, and Sebastian Schelter. 2025. Navigating Data Errors in Machine Learning Pipelines: Identify, Debug, and Learn. In *Companion of the 2025 International Conference on Management of Data, SIGMOD/PODS 2025, Berlin, Germany, June 22-27, 2025*, Volker Markl, Joseph M. Hellerstein, and Azza Abouzied (Eds.). ACM, 813–820. doi:10.1145/3722212.3725636
- [46] Falaah Arif Khan, Denys Herasymuk, Nazar Protsiv, and Julia Stoyanovich. 2025. Still More Shades of Null: An Evaluation Suite for Responsible Missing Value Imputation [Experiment, Analysis and Benchmark]. *Proc. VLDB Endow.* 18, 9 (2025), 2899–2913. doi:10.14778/3746405.3746416
- [47] Mourad Khayati, Alberto Lerner, Zakhar Tymchenko, and Philippe Cudré-Mauroux. 2020. Mind the Gap: An Experimental Evaluation of Imputation of Missing Values Techniques in Time Series. *Proc. VLDB Endow.* 13, 5 (2020), 768–782. doi:10.14778/3377369.3377383
- [48] Kyunghee Kim, Minha Kim, and Simon S. Woo. 2022. STL-DP: Differentially Private Time Series Exploring Decomposition and Compression Methods. In *Proceedings of the CIKM 2022 Workshops co-located with 31st ACM International Conference on Information and Knowledge Management (CIKM 2022), Atlanta, USA, October 17-21, 2022 (CEUR Workshop Proceedings, Vol. 3318)*, Georgios Drakopoulos and Eleanna Kafeza (Eds.). CEUR-WS.org. <https://ceur-ws.org/Vol-3318/short5.pdf>
- [49] Lambert H Koopmans. 1995. *The spectral analysis of time series*. Elsevier.
- [50] Daniel D. Lee and H. Sebastian Seung. 2000. Algorithms for Non-negative Matrix Factorization. In *Advances in Neural Information Processing Systems 13, Papers from Neural Information Processing Systems (NIPS) 2000, Denver, CO, USA, Todd K. Leen, Thomas G. Dietterich, and Volker Tresp (Eds.)*. MIT Press, 556–562. <https://proceedings.neurips.cc/paper/2000/hash/f9d1152547c0bde01830b7e8bd60024c-Abstract.html>
- [51] JiaHao Li, ShuXia Guo, RuLin Ma, Jia He, XiangHui Zhang, DongSheng Rui, YuSong Ding, Yu Li, LeYao Jian, Jing Cheng, et al. 2024. Comparison of the effects of imputation methods for missing data in predictive modelling of cohort study datasets. *BMC Medical Research Methodology* 24, 1 (2024), 41. doi:10.1186/s12874-024-02173-x
- [52] Peng Li, Xi Rao, Jennifer Blase, Yue Zhang, Xu Chu, and Ce Zhang. 2021. CleanML: A Study for Evaluating the Impact of Data Cleaning on ML Classification Tasks. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*. IEEE, 13–24. doi:10.1109/ICDE51399.2021.00009
- [53] Xiao Li, Huan Li, Hua Lu, Christian S. Jensen, Varun Pandey, and Volker Markl. 2023. Missing Value Imputation for Multi-attribute Sensor Data Streams via Message Propagation. *Proc. VLDB Endow.* 17, 3 (2023), 345–358. doi:10.14778/3632093.3632100
- [54] Wei-Chao Lin and Chih-Fong Tsai. 2020. Missing value imputation: a review and analysis of the literature (2006–2017). *Artif. Intell. Rev.* 53, 2 (2020), 1487–1509. doi:10.1007/S10462-019-09709-4
- [55] Jian Liu, Yuansong Qiao, Qi Song, and Yunjun Gao. 2023. Spatio-Temporal Denoising Graph Autoencoders with Data Augmentation for Photovoltaic Data Imputation. In *Companion of the 2023 International Conference on Management of Data, SIGMOD/PODS 2023, New Orleans, LA, USA, May 13-17, 2023*. ACM, 454–466. doi:10.1145/3588730
- [56] Mengna Liu, Pengshuai Yao, Xu Cheng, and Shengyong Chen. 2025. An end-to-end model for time series classification in the presence of missing values. *Expert Systems with Applications* 284 (2025), 127954. doi:10.1016/j.eswa.2025.127954
- [57] Minhao Liu, Ailing Zeng, Muxi Chen, Zhijian Xu, Qiuxia Lai, Lingna Ma, and Qiang Xu. 2022. SCINet: Time Series Modeling and Forecasting with Sample Convolution and Interaction. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/266983d0949aed78a16fa4782237dea7-Abstract-Conference.html
- [58] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wang, Shiyu Wang, Lintao Ma, and Mingsheng Long. 2024. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=JePFA18fah>
- [59] Mohammad Mahdavi and ZiaWasch Abedjan. 2020. Baran: Effective Error Correction via a Unified Context Representation and Transfer Learning. *Proc. VLDB Endow.* 13, 11 (2020), 1948–1961. <http://www.vldb.org/pvldb/vol13/p1948-mahdavi.pdf>
- [60] Rahul Mazumder, Trevor Hastie, and Robert Tibshirani. 2010. Spectral Regularization Algorithms for Learning Large Incomplete Matrices. *J. Mach. Learn. Res.* 11 (2010), 2287–2322. doi:10.5555/1756006.1859931
- [61] Xiaoye Miao, Yangyang Wu, Lu Chen, Yunjun Gao, and Jianwei Yin. 2023. An Experimental Survey of Missing Data Imputation Algorithms. *IEEE Trans. Knowl. Data Eng.* 35, 7 (2023), 6630–6650. doi:10.1109/TKDE.2022.3186498
- [62] Sedir Mohammed, Lukas Budach, Moritz Feuerpfel, Nina Ihde, Andrea Nathansen, Nele Sina Noack, Hendrik Patzlaff, Felix Naumann, and Hazar Harmouch. 2025. The effects of data quality on machine learning performance on tabular data. *Inf. Syst.* 132 (2025), 102549. doi:10.1016/J.IS.2025.102549
- [63] Sedir Mohammed, Felix Naumann, and Hazar Harmouch. 2025. Step-by-Step Data Cleaning Recommendations to Improve ML Prediction Accuracy. In *Proceedings 28th International Conference on Extending Database Technology, EDBT 2025, Barcelona, Spain, March 25-28, 2025*, Alkis Simitsis, Bettina Kemme, Anna Queralt, Oscar Romero, and Petar Jovanovic (Eds.). OpenProceedings.org, 542–554. doi:10.48786/EDBT.2025.43
- [64] Abdenneji Montassar, Tijeni Delleji, Feten Slimeni, Mohamed Ali Abid, and Rabah Attia. 2024. RF Classification of UAVs Using MLP Model and Software Defined Radio. In *21st IEEE/ACS International Conference on Computer Systems and Applications, AICCSA 2024, Sousse, Tunisia, October 22-26, 2024*. IEEE, 1–9. doi:10.1109/AICCSA63423.2024.10912631
- [65] MsMartyPants. 2021. Paris Housing Price Prediction. <https://www.kaggle.com/datasets/mssmartypants/paris-housing-price-prediction> Kaggle dataset.
- [66] Sellers Tracy Talbot Simon Cawthorn Andrew Nash, Warwick and Wes Ford. 1994. Abalone. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C55C7W>
- [67] Felix Neutatz, Binger Chen, ZiaWasch Abedjan, and Eugene Wu. 2021. From Cleaning before ML to Cleaning for ML. *IEEE Data Eng. Bull.* 44, 1 (2021), 24–41. <http://sites.computer.org/debull/A21mar/p24.pdf>
- [68] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. <https://openreview.net/forum?id=Jbdc0vTOcol>
- [69] Curtis G. Northcutt, Lu Jiang, and Isaac L. Chuang. 2021. Confident Learning: Estimating Uncertainty in Dataset Labels. *J. Artif. Intell. Res.* 70 (2021), 1373–1411. doi:10.1613/JAIR.1.12125
- [70] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. 2020. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net. <https://openreview.net/forum?id=r1ecqn4YwB>
- [71] Diane Oyen, Michal Kucer, Nicolas W. Hengartner, and Har Simrat Singh. 2022. Robustness to Label Noise Depends on the Shape of the Noise Distribution. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/e7a217c3389b323fe156046ed3aa1e0e-Abstract-Conference.html
- [72] A. Rajalakshmi and S. S. Sridhar. 2024. Classification of yoga, meditation, combined yoga-meditation EEG signals using L-SVM, KNN, and MLP classifiers. *Soft Comput.* 28, 5 (2024), 4607–4619. doi:10.1007/S00500-024-09695-Y
- [73] Nonparametric Regression. 1992. An Introduction to Kernel and Nearest-Neighbor. *The American Statistician* 46, 3 (1992), 175–185.

- [74] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: Holistic Data Repairs with Probabilistic Inference. *Proc. VLDB Endow.* 10, 11 (2017), 1190–1201. doi:10.14778/3137628.3137631
- [75] Patrick Royston and Ian R White. 2011. Multiple imputation by chained equations (MICE): implementation in Stata. *Journal of statistical software* 45 (2011), 1–20.
- [76] Aryan Sajith and Krishna Chaitanya Rao Kathala. 2024. Is Training Data Quality or Quantity More Impactful to Small Language Model Performance? *CoRR abs/2411.15821* (2024). doi:10.48550/ARXIV.2411.15821 arXiv:2411.15821
- [77] Elena Savelieva, Alexey Kravetski, Sergey Chernov, Vasily V. Demyanov, Vadim Timonin, Rafael V. Arutyunyan, Leonid Aleksandrovich Bolshov, and Mikhail F. Kanevski. 2000. Application of MLP and stochastic simulations for electricity load forecasting in Russia. In *8th European Symposium on Artificial Neural Networks, ESANN 2000, Bruges, Belgium, April 26-28, 2000, Proceedings*. 413–418. <https://www.esann.org/sites/default/files/proceedings/legacy/es2000-502.pdf>
- [78] Tolou Shadbahr, Michael Roberts, Jan Stanczuk, Julian Gilbey, Philip Teare, Sören Dittmer, Matthew Thorpe, Ramon Viñas Torné, Evis Sala, Pietro Lió, et al. 2023. The impact of imputation quality on machine learning classifiers for datasets with missing values. *Communications medicine* 3, 1 (2023), 139.
- [79] Prerna Singh. 2023. Systematic review of data-centric approaches in artificial intelligence and machine learning. *Data Science and Management* 6, 3 (2023), 144–157.
- [80] Daniel J. Stekhoven and Peter Bühlmann. 2012. MissForest - non-parametric missing value imputation for mixed-type data. *Bioinform.* 28, 1 (2012), 112–118. doi:10.1093/BIOINFORMATICS/BTR597
- [81] Michael Stonebraker and El Kindi Rezig. 2019. Machine Learning and Big Data: What is Important? *IEEE Data Eng. Bull.* 42, 4 (2019), 3–7. <http://sites.computer.org/debull/A19dec/p3.pdf>
- [82] Sebastian Strasser and Meike Klettke. 2024. Transparent Data Preprocessing for Machine Learning. In *Proceedings of the 2024 Workshop on Human-In-the-Loop Data Analytics, HILDA 24, Santiago, Chile, 14 June 2024*, Jean-Daniel Fekete, Behrooz Omidvar-Tehrani, Kexin Rong, and Roece Shraga (Eds.). ACM, 1–6. doi:10.1145/3665939.3665960
- [83] Alexander Strehl and Joydeep Ghosh. 2002. Cluster Ensembles – A Knowledge Reuse Framework for Combining Multiple Partitions. *J. Mach. Learn. Res.* 3 (2002), 583–617. <https://jmlr.org/papers/v3/strehl02a.html>
- [84] Pnar Tfekci and Heysem Kaya. 2014. Combined Cycle Power Plant. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5002N>.
- [85] Maria Thurow, Florian Dumpert, Burim Ramosaj, and Markus Pauly. 2021. Goodness (of fit) of imputation accuracy: The GoodImpact analysis. *arXiv preprint arXiv:2101.07532* (2021).
- [86] Ilya O. Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner, Daniel Keysers, Jakob Uszkoreit, Mario Lucic, and Alexey Dosovitskiy. 2021. MLP-Mixer: An all-MLP Architecture for Vision. In *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan (Eds.). 24261–24272. <https://proceedings.neurips.cc/paper/2021/hash/cba0a4ee5cc02fda0fe3f9a3e7b89fe-Abstract.html>
- [87] Hugo Touvron, Piotr Bojanowski, Mathilde Caron, Matthieu Cord, Alaeldin El-Nouby, Edouard Grave, Gautier Izacard, Armand Joulin, Gabriel Synnaeve, Jakob Verbeek, and Hervé Jégou. 2023. ResMLP: Feedforward Networks for Image Classification With Data-Efficient Training. *IEEE Trans. Pattern Anal. Mach. Intell.* 45, 4 (2023), 5314–5321. doi:10.1109/TPAMI.2022.3206148
- [88] Vladimir N Vapnik and A Ya Chervonenkis. 2015. On the uniform convergence of relative frequencies of events to their probabilities. In *Measures of complexity: festschrift for alexey chervonenkis*. Springer, 11–30.
- [89] Huiqiang Wang, Jian Peng, Feihu Huang, Jince Wang, Junhui Chen, and Yifei Xiao. 2023. MICN: Multi-scale Local and Global Context Modeling for Long-term Series Forecasting. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. <https://openreview.net/forum?id=zt53IDUR1U>
- [90] Ruilin Wang, Kele Chi, Yanghao Wang, Quoc Viet Hung Nguyen, Hongzhi Yin, and Xiaofang Zhou. 2024. In-Database Data Imputation. *Proceedings of the ACM on Management of Data* 2, 1, Article 111 (2024). doi:10.1145/3639326
- [91] Shiyu Wang, Haixu Wu, Xiaoming Shi, Tengge Hu, Huakun Luo, Lintao Ma, James Y. Zhang, and Jun Zhou. 2024. TimeMixer: Decomposable Multiscale Mixing for Time Series Forecasting. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=7oLshfEIC2>
- [92] Yuxuan Wang, Haixu Wu, Jiaxiang Dong, Yong Liu, Mingsheng Long, and Jianmin Wang. 2024. Deep Time Series Models: A Comprehensive Survey and Benchmark. *CoRR abs/2407.13278* (2024). doi:10.48550/ARXIV.2407.13278 arXiv:2407.13278
- [93] Steven Euijong Whang, Yuji Roh, Hwanjun Song, and Jae-Gil Lee. 2023. Data collection and quality challenges in deep learning: a data-centric AI perspective. *VLDB J.* 32, 4 (2023), 791–813. doi:10.1007/S00778-022-00775-9
- [94] Katarzyna Woznica and Przemyslaw Biecek. 2020. Does imputation matter? Benchmark for predictive models. *CoRR abs/2007.02837* (2020). arXiv:2007.02837 <https://arxiv.org/abs/2007.02837>
- [95] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2023. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/forum?id=ju_Uqw384Oq
- [96] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2021. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. In *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan (Eds.). 22419–22430. <https://proceedings.neurips.cc/paper/2021/hash/bcc0d400288793e8bdcd7c19a8ac0c2b-Abstract.html>
- [97] Lingfeng Yang, Xiang Li, Renjie Song, Borui Zhao, Juntian Tao, Shihao Zhou, Jiajun Liang, and Jian Yang. 2022. Dynamic MLP for Fine-Grained Image Classification by Leveraging Geographical and Temporal Information. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*. IEEE, 10935–10944. doi:10.1109/CVPR52688.2022.01067
- [98] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. 2025. LIMO: Less is More for Reasoning. *CoRR abs/2502.03387* (2025). doi:10.48550/ARXIV.2502.03387 arXiv:2502.03387
- [99] I-Cheng Yeh. 1998. Concrete Compressive Strength. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5PK67>.
- [100] Kun Yi, Qi Zhang, Wei Fan, Shoujin Wang, Pengyang Wang, Hui He, Ning An, Defu Lian, Longbing Cao, and Zhendong Niu. 2023. Frequency-domain MLPs are More Effective Learners in Time Series Forecasting. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/f1d16af76939f476b5f040fd1398c0a3-Abstract-Conference.html
- [101] Jinsung Yoon, James Jordon, and Mihaela van der Schaar. 2018. GAIN: Missing Data Imputation using Generative Adversarial Nets. In *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholm, Sweden, July 10-15, 2018 (Proceedings of Machine Learning Research, Vol. 80)*, Jennifer G. Dy and Andreas Krause (Eds.). PMLR, 5675–5684. <http://proceedings.mlr.press/v80/yoon18a.html>
- [102] Li Yuan and Xue-Yi Zhao. 2025. A Hybrid MLP and CNN Architecture for Sequential Recommendation. *IEEE Access* 13 (2025), 91515–91523. doi:10.1109/ACCESS.2025.3569820
- [103] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are Transformers Effective for Time Series Forecasting? In *Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023*, Brian Williams, Yiling Chen, and Jennifer Neville (Eds.). AAAI Press, 11121–11128. doi:10.1609/AAAI.V37I9.26317
- [104] Daochen Zha, Zaid Pervaiz Bhat, Kwei-Herng Lai, Fan Yang, Zhimeng Jiang, Shaochen Zhong, and Xia Ben Hu. 2025. Data-centric Artificial Intelligence: A Survey. *ACM Comput. Surv.* 57, 5 (2025), 129:1–129:42. doi:10.1145/3711118
- [105] Aoqian Zhang, Shaouxu Song, Yu Sun, and Jianmin Wang. 2019. Learning Individual Models for Imputation. In *35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8-11, 2019*. IEEE, 160–171. doi:10.1109/ICDE.2019.00023
- [106] Jiaxuan Zhang, Haitao Yuan, Jianing Si, Nan Jiang, and Shangguang Wang. 2025. Think Twice Before Imputation: Optimizing Data Imputation Order for Machine Learning. In *41st IEEE International Conference on Data Engineering, ICDE 2025, Hong Kong, May 19-23, 2025*. IEEE, 1362–1374. doi:10.1109/ICDE65448.2025.00106
- [107] Tianping Zhang, Yizhuo Zhang, Wei Cao, Jiang Bian, Xiaohan Yi, Shun Zheng, and Jian Li. 2022. Less Is More: Fast Multivariate Time Series Forecasting with Light Sampling-oriented MLP Structures. *CoRR abs/2207.01186* (2022). doi:10.48550/ARXIV.2207.01186 arXiv:2207.01186
- [108] Yuwei Zhang, Weiwei Deng, Haiwei Zhang, Shaouxu Song, and Shuo Shang. 2023. DAMR: Dynamic Adjacency Matrix Representation Learning for Multivariate Time Series Imputation. In *Companion of the 2023 International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 589–601. doi:10.1145/3589333
- [109] Yimei Zhang, Xiangjie Kong, Wenfeng Zhou, Jin Liu, Yanjie Fu, and Guojiang Shen. 2024. A Comprehensive Survey on Traffic Missing Data Imputation. *IEEE Trans. Intell. Transp. Syst.* 25, 12 (2024), 19252–19275. doi:10.1109/TITS.2024.3478816
- [110] Cheng Zhen, Amandeep Singh Chabada, and Arash Termehchy. 2023. When Can We Ignore Missing Data in Model Training? In *Proceedings of the Seventh Workshop on Data Management for End-to-End Machine Learning, DEEM 2023, Seattle, WA, USA, 18 June 2023*. ACM, 4:1–4:4. doi:10.1145/3595360.3595854

- [111] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. In *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*. AAAI Press, 11106–11115. doi:10.1609/AAAI.V35I12.17325
- [112] Yuhao Zhou, Fengjiao Tu, Kewei Sha, Junhua Ding, and Haihua Chen. 2024. A Survey on Data Quality Dimensions and Tools for Machine Learning Invited Paper. In *IEEE International Conference on Artificial Intelligence Testing, AITest 2024, Shanghai, China, July 15-18, 2024*. IEEE, 120–131. doi:10.1109/AITest62860.2024.00023